



GlycoMSP User Manual

v1.0

GUI version v1.10 • 2026/06/28

Contents

GlycoMSP User Manual	3
How to read this manual	3
Part 1 — Introduction	3
1.1 What GlycoMSP does	3
1.2 Where GlycoMSP fits	4
1.3 The two paths at a glance	4
1.4 What you'll need	4
Part 2 — Quickstart: your first end-to-end run via the CGA pipeline	5
2.1 Launch GlycoMSP and meet the main window	5
2.2 Add raw files and convert to CSV	6
2.3 Open Prepare Dataset and add your scan CSV	9
2.4 Run CGA Manager — set up the in-silico library and Score A inputs	10
2.5 Start CGA analysis and produce the pseudolabel TSV	12
2.6 Build the trainable CSV (CGA → Trainable)	12
2.7 Train a Random Forest model	14
2.8 Run prediction on unlabeled data	18
2.9 Read your outputs	19
Part 3 — Workflow guides	20
3.1 The MAS pipeline with expert annotations	20
3.2 Combining trainable datasets (and when not to)	21
3.3 Score B enrichment — motif-aware re-ranking	22
3.4 Negative sampling options	23
3.5 Ion mining — discovering diagnostic fragments	24
3.6 Building unlabeled datasets across multi-sample experiments	25
Part 4 — Concepts	26
4.1 Score A — ion-fitting score	26
4.2 Score B — motif-aware re-ranking	27
4.3 Feature extraction	27
4.4 The ML model — Random Forest	28
4.5 Reproducibility — training_run.json	28
4.6 Why two parallel labeling pipelines?	29
4.7 Ion mining (terminology)	29
4.8 Evaluating MAS-ML: within-tissue, leave-one-tissue-out, and pooled CV	29
Part 5 — Reference	30
5.1 Input file formats and column requirements	30
5.2 Method JSON v1 schema	31
5.3 Experiment JSON schema	33
5.4 Output file specifications	34
5.5 Default parameters	36
5.6 Score B workbook structure (overview)	37
Part 6 — Configuration & advanced	38
6.1 The ML parameters editor	38
6.2 Precursor (MS1) gate	40
6.3 Threshold sweeps and τ selection	40
Part 7 — Troubleshooting & known limitations	41
7.1 Common errors and their fixes	41

7.2 Known limitations	42
Part 8 — Reproducing the manuscript results	43
8.0 Quick start for reviewers	44
8.1 Setup and environment	44
8.2 MAS-ML evaluation — Result 2 (Supp S2c ID 7)	44
8.3 CGA / CGA-ML — Results 3-4 (Supp S2c ID 4c, 1c)	44
8.4 External-tool comparison — Result 5 (Supp S2c ID 8-9, Supp Table S3)	45
Appendix A — Glossary	45
Appendix B — Keyboard and right-click reference	47
Dataset Explorer (Prepare Dataset window)	47
Modal dialog conventions	48
Main window	48
Ion Suggestions Viewer	48
Appendix C — Pointers to related documents	48

GlycoMSP User Manual

Version: v1.0 **Tool version covered:** GlycoMSP v1.10 (mspfileloaderv14.py) **Last updated:** 2026-06-28 **Audience:** Users (chemists and glycobiologists), thesis defense committee, manuscript reviewers

How to read this manual

This manual is built so different readers can enter at different points and not waste time.

If you're a **chemist or glycobiologist** running your own samples through the tool, start at **Part 2 — Quickstart** and follow it linearly. Part 1 gives you the conceptual picture in two minutes; Part 4 explains *why* the scoring works the way it does if you're curious.

If you're on the **thesis committee** and want a structural picture of the system, read **Part 1** for the pipeline orientation, then jump to **Part 4 — Concepts** for the rationale behind the two annotation routes and the scoring design.

If you're a **manuscript reviewer** evaluating reproducibility, read **Part 1** for orientation, then go to **Part 8 — Reproducing the manuscript results**, which maps each result to its Zenodo package with exact commands and expected numbers. **Part 5 — Reference** lists default parameters, JSON schemas, and output file specifications, and the **Concepts** section (Part 4, including §4.8 on the MAS-ML evaluation design) is where the manuscript-aligned formulas and evaluation rationale live.

A glossary of mass-spectrometry and machine-learning terms is in **Appendix A**. The manual assumes you already work with mass spectrometry; machine-learning terminology is defined in-prose on first use and again in the glossary.

Part 1 — Introduction

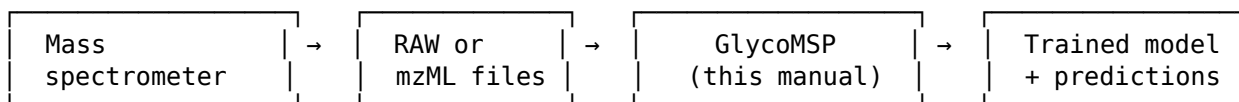
1.1 What GlycoMSP does

GlycoMSP is a desktop application that converts glycomics tandem mass spectrometry (MS2) data into machine-learning-ready training matrices, trains classifiers on those matrices, and applies the trained models to predict glycan compositions for unlabeled spectra.

It supports two complementary paths to a labeled training set. The first, **Constraint-based Glycan Annotation (CGA)**, generates labels computationally from in-silico libraries and an ion-evidence scoring scheme; this path is appropriate when no expert-validated annotations exist. The second, **Manual annotation support (MAS)**, ingests labels from an expert-prepared annotation Excel sheet and validates them against the spectra; this path is appropriate when ground-truth annotations are available. The two paths converge on the same feature representation, so models can be trained on data from either path or a mixture of both.

Around the two annotation paths, the application provides supporting workflows: per-RAW-file metadata capture and conversion, optional negative-class (Non-glycan) sampling, optional motif-aware re-ranking via Score B, ion-mining for diagnostic-fragment discovery, and a prediction step that produces ranked predictions with confidence scores plus reviewer-friendly output reports.

1.2 Where GlycoMSP fits



Upstream, GlycoMSP is fed by any LC-MS/MS instrument that produces RAW (Thermo) or mzML files. On Windows with the Thermo MSFileReader library installed, GlycoMSP reads .raw directly; on macOS, Linux, or Windows machines without MSFileReader, you supply pre-converted .mzML files.

Downstream, GlycoMSP produces three artifact families: (1) a trained classifier saved as .skops (with .joblib and .pkl fallbacks accepted on load), (2) prediction CSVs accompanied by a class-summary file and a JSON/Markdown run summary, and (3) review queues highlighting spectra that fell below confidence thresholds for human inspection.

1.3 The two paths at a glance

	CGA pipeline (computed labels)	MAS pipeline (expert labels)
When to use	You have raw spectra but no expert annotations	You have expert annotations validated against the spectra
You provide	RAW/mzML, sample metadata, configuration flags for the in-silico library generator	RAW/mzML, sample metadata, an annotation Excel (MSlist + ionlist sheets)
GlycoMSP generates	An in-silico glycan library, candidate compositions per spectrum, Score A (and optionally Score B), per-spectrum top-ranked composition labels	A merged annotation table linking expert labels to spectra, with optional automatic Non-glycan negatives
Output	ML-ready wide CSV with constraint-based labels	ML-ready wide CSV with expert-verified labels
Mixable for training?	Yes — both produce the same column schema	Yes

The two paths converge after labeling: feature extraction (log-transformed intensity at PPM-matched fragment masses) is identical, and the ML training and prediction steps make no distinction between CGA-derived and MAS-derived rows. Manuscript Methods uses the names **CGA-ML** and **MAS-ML** for the end-to-end pipelines built around each labeling path.

1.4 What you'll need

Before starting, gather the following:

- **One or more .raw or .mzML files** with MS2 fragmentation data. CGA and MAS both require MS2 spectra. To avoid confusion, each conversion batch must be **either all RAW or all mzML** — a single batch does not mix the two formats.
- **Sample metadata** for each RAW file: experiment title, glycan type (N-glycan or O-GalNAc), mass-analyzer charge mode (positive or negative), derivatization chemistry, and (for negative mode) any modifier-class flags. GlycoMSP will prompt you for these the first time it sees a file; subsequent files in the same batch can reuse the previous metadata as a starting point.
- **An ion list (Excel or CSV)** with reference fragment m/z values and glycotope/structural identifiers. This is required for Score A computation in the CGA pipeline and for fragment-feature extraction. The same ion list is reusable across samples sharing the same derivatization and ionization mode.

- **An expert annotation Excel** — only for the MAS pipeline. Must contain an MSlist sheet (scan IDs and glycan labels) and an ionlist sheet (reference fragment ions). See Part 5.1 for the exact column requirements.
- **A Score B workbook** (optional). A six-sheet Excel file encoding glycotopic motif rules and penalty parameters; only needed if you wish to enable motif-aware re-ranking. By default Score B is disabled, and Score A alone drives candidate selection. See Part 3.3 for an overview and `score_b_config_instructions.md` (in the manuscript supplementary materials) for full configuration.

Two further notes on the operating environment. First, file paths inside saved JSON method files are stored as POSIX-style for the experiment file and native (OS-dependent) for the per-sample method files; the manual conversion is handled internally and you do not need to think about it. Second, the GUI title bar shows the build version (e.g., GlycoMSP File Manager GUI v1.10 ...); cite this when reporting issues.

Part 2 — Quickstart: your first end-to-end run via the CGA pipeline

This Quickstart walks one realistic end-to-end run through the **CGA pipeline** (constraint-based glycan annotation): from raw instrument files all the way to a trained model and predictions on unlabeled data. We chose CGA because it does not require pre-existing expert annotations; everything you need to label your data is generated by the tool. The MAS pipeline (Part 3.1) and Score B enrichment (Part 3.3) build on the same backbone and only diverge at specific steps.

Conventions used here. Italicised words are screen labels (button text, field labels) reproduced verbatim. Code-style text refers to file names, column names, or values you should literally type. Curly braces like {this} mark placeholders you replace with your own values. Glycan composition labels are written **fuco****se-first** in the fixed order F (Fuc) → H (Hex) → N (HexNAc) → S (Neu5Ac) → G (Neu5Gc) → KDN, with a count after each present monosaccharide (e.g., F1H5N4 is the bi-antennary composition with 1 fucose, 5 hexoses, and 4 N-acetylhexosamines; F1H5N5KDN2 carries 2 KDN). KDN is written as the literal token KDN, not K.

2.1 Launch GlycoMSP and meet the main window

Launch GlycoMSP by running `python mspfileloaderv14.py` from the project directory, or by double-clicking the build's launcher on Windows. The first thing you see is the **main window**:

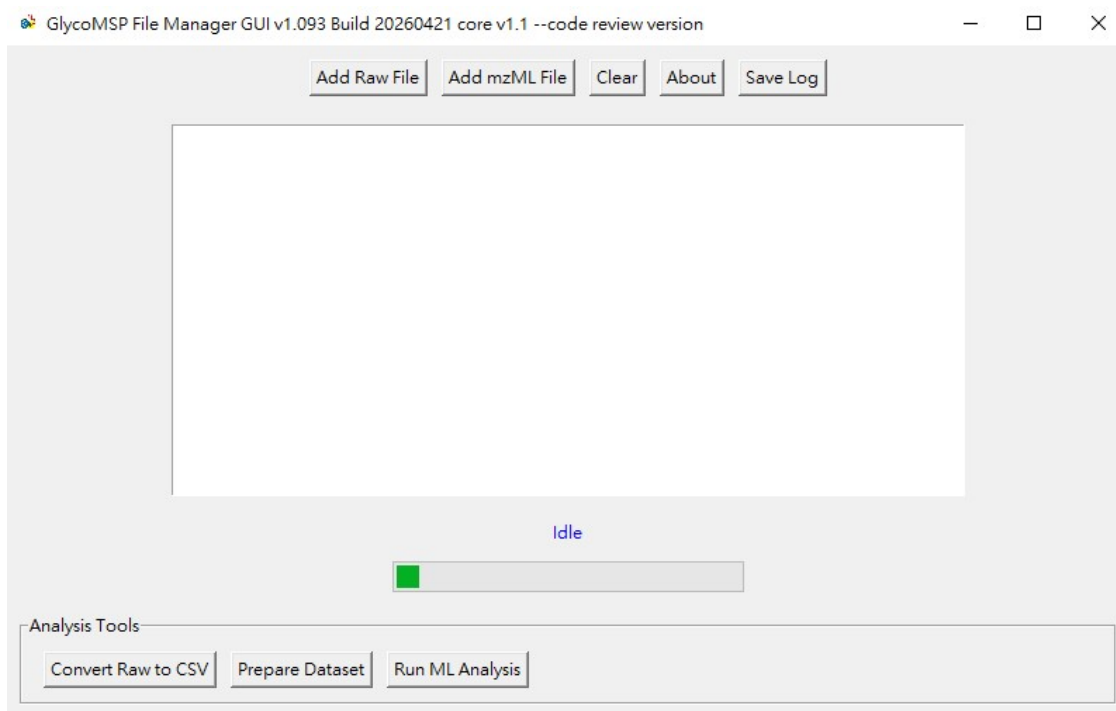


Figure 1. Main launcher window

Title bar shows the build version. Top button row (left to right): *Add Raw File*, *Add mzML File*, *Clear*, *About*, *Save Log*. Centre is a large white text panel — this is the **log** where the application prints status messages as it works. Below the log is a status indicator (showing *Idle* in blue with a small green progress bar). Bottom of the window is a labelled section titled *Analysis Tools* containing three primary action buttons: *Convert Raw to CSV*, *Prepare Dataset*, *Run ML Analysis*.

The three *Analysis Tools* buttons map directly to the three stages of a GlycoMSP run. You'll work them left to right: **convert** raw instrument files to scan CSVs, **prepare** a labeled training dataset from those CSVs, then **run ML analysis** to train a model and use it to make predictions.

The top-row file-add buttons (*Add Raw File* / *Add mzML File*) populate a working list of files for batch conversion. *Clear* empties this list. *About* opens version and module information. *Save Log* writes the contents of the centre log panel to a `.log` file.

2.2 Add raw files and convert to CSV

Click **Add Raw File** (Windows) or **Add mzML File** (macOS / Linux / pre-converted) and select one or more files. Selected paths are appended to the application's working list and shown in the log panel.

Now click **Convert Raw to CSV** in the *Analysis Tools* section. This opens the **Metadata Editor** window for the first file in your list.

Converting: zf_sPerMeNG_brain.raw

Converting raw file... please wait.

Experiment Title

Experiment Description

Author running this analysis

Raw data acquired date

Glycan Type

Mass Analyzer charge mode

Derivatization Type

Raw File (optional):

Figure 2. Metadata Editor window (zebrafish brain example)

Figure 3. Metadata Editor window (zebrafish brain example) (continued)

A modal dialog with a header showing the current file name. The form has seven fields: *Experiment Title* (Combobox), *Glycan Type* (Combobox: N-glycan / O-GalNAc), *Mass Analyzer charge mode* (Combobox: positive / negative), *Reduced* (Combobox), *Derivatization* (Entry), *Sample Type* (Entry), and *Operator* (Entry). At the bottom is a five-button bar: *Generate Dataset*, *Import Metadata*, *Use Last Metadata*, *Clear*, and an optional *Browse...* for the raw file path. Status label at the very bottom shows conversion progress in colour (blue = converting, green = idle, red = error, orange = batch waiting).

Fill in the metadata for this first file. The three Combobox fields and *Derivatization* are required; the others are reserved for record-keeping. If you have a previous metadata JSON saved from an earlier run, click **Import Metadata** to populate the fields from it. After the first file in a batch, **Use Last Metadata** copies the previous file's metadata as a starting point — useful when a batch shares experimental conditions.

When the form is complete, click **Generate Dataset**. The application:

1. Runs the raw-to-CSV (or mzML-to-CSV) conversion in a background thread while the form locks. The log panel and status bar show progress.
2. Writes a JSON metadata sidecar (.raw.json or .mzml.json) next to the converted CSV.
3. Promotes the temporary scan CSV to its final location.
4. Advances to the next file in the batch and re-opens the Metadata Editor with *Use Last*

Metadata pre-populated, or closes the editor when the batch is complete.

The resulting **scan CSV** is the input to everything else. One row per MS2 scan, with the key columns MS2scan_no, protonatedmass, peaklist, and peakintensity (plus MS1 precursor/charge fields such as MS1scan_no, MS1_isolationmass, MS1_monoisolationmass, and chargeState).

Note. GlycoMSP focuses on MS2 scan-level traceability tied to the peak list and intensities; retention time and other LC/EIC-related parameters are not retained in the current design.

If a conversion fails (corrupt file, missing decoder library, unwritable output directory), the editor enters an error state: the status label turns red, the window title prefixes “Error:”, and the rest of the batch is halted. Use **Save Log** on the main window to capture the diagnostic output before closing.

Reporting conversion errors. GlycoMSP’s MS2-indexed conversion has so far been tested specifically on Thermo RAW and their corresponding mzML spectra. If you hit an error, we would appreciate a report so we can improve the reliability of the initial MS2-indexed dataset preparation. Please include: the **error log**; **screenshots** (optional); the **file(s) that failed** (optional); and a short description of the spectral data — **vendor, instrument, and acquisition mode (DDA / DIA / SRM or other specific settings)**.

2.3 Open Prepare Dataset and add your scan CSV

Back at the main window, click **Prepare Dataset**. This opens the **Prepare Dataset** window, which is where the CGA labeling work happens.



Figure 4. Prepare Dataset window

Top-left: a *Check integrity* button (currently a placeholder for a future feature). Below it: the **Dataset Explorer** tree, a hierarchical view that will hold your ex-

periments and samples. Centre: the action button grid, organised into rows. Row 1 — *Add MS2 CSV(s), Add Excels (ion list / man annotation), Add Sample using metadata*. Row 2 — *Add Sample, Clean up empty unassigned sample tags, Link Sample, Merge Sample*. Row 3 — *Negative Sampling Setting, Ion Mining, Browse Ion Mining result, Load Method*. Row 4 — *CGA manager, CGA → Trainable*. Below the action grid: a *Close* button. Status footer: *Experiment method file:* and *Sample method will be saved at:.* Bottom row: *Set Experiment Method File, Set Sample Method Folder, Load .exp.json, Save .exp.json, Export Method v1 (MAS), Export Method v1 (CGA)*.

Click **Add MS2 CSV(s)** and select the scan CSV(s) you produced in Step 2.2. They appear in the Dataset Explorer under a top-level node called *Unassigned*, with one *Sample* sub-node per file. The sample names default to the CSV filename stem.

If you'd rather group samples under a meaningful experiment name from the start, you can instead click **Add Sample using metadata** and select your metadata JSON files. GlycoMSP reads the *Experiment Title* and *Raw filename* fields and creates the experiment node and sample node accordingly. (Internally these two paths just call different file-handler routines; the resulting tree structure is identical.)

The Dataset Explorer is your working surface for the rest of the workflow. Right-clicking a sample exposes per-sample actions, and drag-and-drop is available, but the tree view is not fully reliable — not every click is guaranteed to register, so treat it as a convenience rather than a dependable control surface.

Note. The validation status indicator (“OK” = linked-and-validated; “warning” / “error” = issues) is only updated in the **MAS** workflow, where it gates the *Link/Validate* and *Merge* buttons. The CGA workflow does not refresh this indicator in the tree; this is a minor display-only issue and is safe to ignore, as the indicator does not guard the CGA path.

2.4 Run CGA Manager — set up the in-silico library and Score A inputs

Select your sample (single-click in the tree), then click **CGA manager**. This opens the **CGA Setup** window.

Figure 5. CGA Setup window (zebrafish brain library settings)

A modal dialog (~900×800). Top section: three path fields — *In-silico CSV*, *Ion List*, *Score B workbook* (optional) — each with associated *Add*, *Clear*, and (for Score B) *Validate* buttons. Middle section: metadata read-back display showing *Experiment Title*, *Glycan Type*, *Mass Analyzer charge mode* with a *Reset from file* button. Below that: a flag panel split into *Left* and *Right* columns containing checkboxes (e.g., *hybrid*, *bisecting*, *poly-LacNAc*), spinboxes (e.g., *arm count* range), and core-type selection (for O-GalNAc only). Bottom: *Advanced* toggle, action buttons: *Generate In-Silico CSV*, *Link in silico glycan list*, *Save CGA settings*, *Load CGA settings*, and *Start CGA analysis*.

This window has three tasks: (a) generate or link an in-silico glycan library, (b) attach an ion list, and (c) optionally attach and validate a Score B workbook.

(a) In-silico library. If you don't have a library file already, configure the flag panel for your glycan class (the fields shown depend on whether *Glycan Type* is N-glycan or O-GalNAc) and click **Generate In-Silico CSV**. The application calls the composition generator (compv4.NGlaunch for N-glycans or compv4.0Glaunch for O-glycans), produces a CSV listing every theoretical composition consistent with your flags and modifier counts, and stores its path in the *In-silico CSV* field. The library is **reusable across samples** that share derivatization and ionization mode; if you already generated one, click **Link in silico glycan list**

instead and pick the file directly.

(b) Ion list. Click **Add Ion List** and pick your ion-list Excel or CSV. The file should contain one row per reference fragment with at minimum a mass column (recognised header variants: mass, mz, ion_mz, m/z, fragmentation_mass) and a structural identifier column (recognised: structural_identifier, glycotope_name, glycotope, fragment_name, fragment, ion_name, annotation, name, label, in priority order).

(c) Score B workbook (optional). If you want motif-aware re-ranking, click **Add Score B Workbook** and pick your six-sheet Excel file (.xlsx, .xslm, .xltx, or .xltn). Then click **Validate Score B Workbook** to run a quick load test; the application will pop up a confirmation or an error message naming the offending sheet. **Skip this for your first run** — Score B is disabled by default, and the CGA pipeline produces useful results from Score A alone. Part 3.3 covers Score B in depth.

2.5 Start CGA analysis and produce the pseudolabel TSV

With the in-silico library and ion list attached (Score B is optional), click **Start CGA analysis**. The application:

1. Loads the in-silico library and normalises composition columns.
2. For each MS2 scan, performs binary-search precursor mass matching against the library at your configured PPM tolerance (default 10 ppm), producing candidate compositions per scan.
3. For each candidate, attaches an ion-fitting score (Score A) by counting how many reference fragments match peaks in the spectrum within tolerance, gated by an “anchor” requirement (a minimum number of high-intensity required ions must be present).
4. If a Score B workbook was attached and validated, runs the Score B enrichment pass: each candidate is re-evaluated against motif-evidence rules and assigned a Score B value $b \in [0, 1]$. Score B failure is non-fatal — the original Score A TSV is preserved.
5. Writes the result to a tab-separated file (the **pseudolabel TSV**) with columns including MS2scan_no, comp_str, comp_tuple, ion_score, ion_hit_count, ion_hits_mz, and (when Score B is active) score_b, score_b_rank, score_b_selected, score_b_motif_summary.

The pseudolabel TSV is your raw labeling output: one row per (scan, candidate composition) pair. The next step collapses this to one row per scan.

2.6 Build the trainable CSV (CGA → Trainable)

Back in the Prepare Dataset window with your sample selected, click **CGA → Trainable**. This opens the *CGA → Trainable (one-pass)* modal.

Figure 6. CGA → Trainable modal

A modal dialog. Path fields at top (pseudolabel TSV path, ion-list path, optional salvage path, optional wide-output override) each with a *Browse...* button. Below: thresholds section — minimum ion score, top-N candidates per scan. Middle: Negatives section — checkbox to enable Non-glycan sampling, ratio control, hits gate. Below: Feature Mode section — radio buttons for *Rebuild features from peak list* (recompute) versus *Reuse pre-computed features* (faster). Below: Mass Feature checkbox (*Use protonated mass as a model feature; Non-glycan = 0*). Bottom: *Build Trainable CSV* button + status line.

For your first run, accept the defaults. Confirm the **pseudolabel TSV** field points at the file from Step 2.5 (it should auto-fill from the sample's files["CGAresult_tsv"] slot) and that **ion list** points at the file from Step 2.4. Leave the negatives checkbox enabled and the feature mode at *Rebuild*.

Click **Build Trainable CSV**. The application:

1. Reads the pseudolabel TSV and selects the top-ranked candidate per scan. If Score B is active, ranking uses score_b_rank; otherwise it uses ion_score with your minimum threshold.
2. Optionally identifies low-evidence scans as Non-glycan negatives via `ml_ng_utils.collect_ng_candidates`. By default, scans with very low Score A and few or no ion hits become candidates for the *Non-glycan* class.
3. Builds the feature matrix. Each spectrum becomes one row; each reference fragment in the ion list becomes one column. Cell values are $i_feature, n = \log_{10}(I_n) + 1$ where I_n is the maximum intensity within the ion-matching tolerance of fragment n 's mass (default 10 ppm for training-set feature encoding), or 1.0 if no peak matched. The transformation compresses the dynamic range while keeping zero-intensity hits distinguishable from missing matches.
4. Writes the **trainable CSV** with columns MS2scan_no, [ion_mz_1, ion_mz_2, ..., ion_mz_N], and Structure (the composition label, or the literal string Non-glycan for negatives).

The status line shows total scans processed, positive-class scans selected, and (if enabled)

negatives added. When the run completes, the trainable CSV path is recorded in the sample's files["trainable_csv"] slot, and the path is also written into the per-sample method JSON.

You now have one trainable CSV per processed sample, ready for ML training. If you have multiple samples, you can train per-sample or combine them; combining is covered in Step 2.7.

2.7 Train a Random Forest model

Close the Prepare Dataset window (your work is auto-saved into the per-sample method JSON if a method folder is set, or held in the in-memory experiment state otherwise; see Part 5.3 for details). Back at the main window, click **Run ML Analysis**. The window opens on the **Train Model** tab.

ML Analysis

Train Model Predict

Step 1: Load Trainable Dataset (.csv) Select CSV File

☐ CGA dataset? ☐ Use protonated mass as a model feature (Non-glycan = 0)

Step 2: Select Label Column Structure

Step 3: Choose Classifier Random Forest

Step 4: Train/Test Parameters Set Parameters / Train the Model

Params: (using built-ins)

Train Model

Trainable File Info (Origin Tracking)

Combine Datasets for Training Select Datasets

Close

Figure 7. ML Analysis window, Train Model tab

Title bar: *ML Analysis*. Two tabs at the top: *Train Model* (active) and *Predict*. Step-numbered form below: *Step 1: Load Trainable Dataset (.csv)* with a *Select CSV File* button, plus two checkboxes — *CGA dataset?* and *Use protonated mass as a model feature (Non-glycan = 0)*. *Step 2: Select Label Column* combobox (default *Structure*). *Step 3: Choose Classifier* combobox (default *Random Forest*). *Step 4: Train/Test Parameters* — *Set Parameters / Train the Model* button + *Params:* status line (default (using built-ins)). Below: a large blue *Train Model* button. Below that: a *Trainable File Info (Origin Tracking)* read-only panel and a *Combine Datasets for Training* row with *Select Datasets* button. *Close* button at the bottom.

Step 1 — Load. Click **Select CSV File** and pick the trainable CSV from Step 2.6. The *Trainable File Info* panel populates with the file path, row count, column count, and class distribution. Tick **CGA dataset?** so the application records the labeling origin in the training-run metadata. Tick **Use protonated mass as a model feature (Non-glycan = 0)** to add the precursor mass as an input feature (recommended for CGA-derived datasets — mass agreement is part of how the labels were assigned, so the model benefits from seeing it explicitly; Non-glycan rows have their precursor mass set to zero).

Step 2 — Label column. Leave at *Structure* unless your CSV uses a different column name.

Step 3 — Classifier. Leave at *Random Forest*. (Other classifiers can be added in future versions; the current implementation ships a single, well-tuned Random Forest pipeline.)

Step 4 — Parameters. Click **Set Parameters / Train the Model**. This opens the **Train/Test Parameters** dialog, where you set the train/validation/test split, class-balancing options, and Random Forest hyperparameters.

Train/Test Parameters

Test split

0.20

Validation split

0.00

Note: Test + Validation must be < 1.0

Train 80% | Val 0% | Test 20%

Minimum samples per class

5

☒ Enable class balancing (cap majority)

Majority label: Non-glycan

Majority factor ($\times$ max minor): 3

☒ Stratify by label

Random Forest Options

n_estimators (trees): 400

☒ Use class_weight = "balanced"

☒ Real-world test (cap training only)

Prediction Thresholding

☐ Enable confidence threshold $\rightarrow$ fallback to Majority label

Threshold τ (0.00–0.99): 0.65

Majority support margin δ (0.00–0.20): 0.05

Edit ML Parameters...

Force current params

Close

Figure 8. Train/Test Parameters dialog

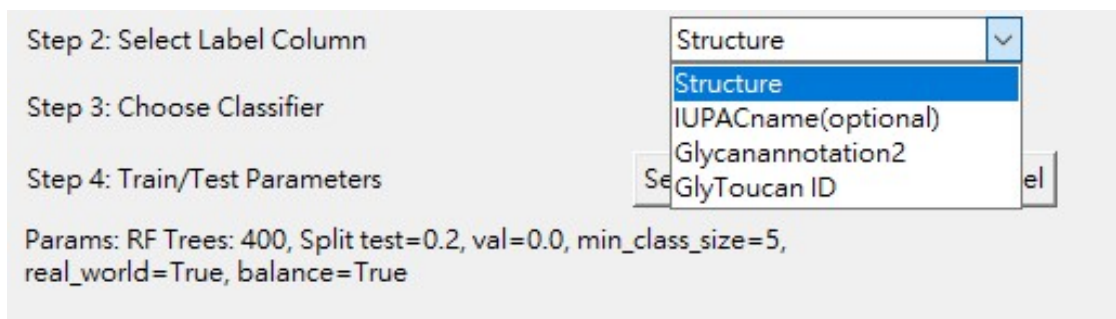


Figure 9. Train/Test Parameters dialog (continued)

Modal dialog. Sliders for *Test split* (default 0.20) and *Validation split* (default 0.00) with a live percentage readout (e.g., *Train 80% | Test 20%*). *Min samples per class* spinbox (default 5). Class-balancing block: *Enable balancing* checkbox + *Cap value* spinbox + *Cap training only* checkbox (preserves the natural distribution in the held-out test). *Stratify* checkbox (default on). RF block: *n_estimators* (default 400), *class_weight* (default *balanced*). Threshold block: *Apply confidence threshold* checkbox (**off by default**), *Tau* (default 0.65), *Margin* (default 0.05). Bottom: *Apply* button.

For your first run, accept the defaults. The v1.10 defaults are: an 80/20 train/test split (`test_split = 0.20`, `val_split = 0.00`; no validation fold by default), stratified split, balanced class weights, 400 trees in the Random Forest (`max_depth=None`, `min_samples_split=2`, `min_samples_leaf=1`, `random_state=42`), a class-eligibility filter that drops compositions with fewer than 5 spectra, and the Non-glycan class capped at 3× the largest glycan class on the training fold only (the held-out test is left uncapped). The optional confidence-threshold step ($\tau = 0.65$, `margin = 0.05`) is **disabled by default**; when enabled, it demotes predictions below τ (or within margin of the next class) into a “low-confidence” review queue at prediction time. Click **Apply**.

Note — fixed (non-configurable) RF parameters. `max_depth=None`, `min_samples_split=2`, `min_samples_leaf=1`, and `random_state=42` are hard-coded in v1.10 and cannot be changed from the GUI. Only `n_estimators` and `class_weight` are exposed in the dialog. The dialog closes and the *Params* status line in the Train Model tab updates from (using built-ins) to a short summary.

Train. Click the large blue **Train Model** button. The pipeline:

1. Imports the trainable CSV and drops administrative columns (`uid`, `origin_*`, `predicted_label` if present) to produce the numeric feature matrix.
2. If *Use protonated mass as a model feature* was ticked, sets `protonatedmass = 0` for Non-glycan rows (string-label match) before label encoding.
3. Drops classes with fewer than *Min samples per class* rows.
4. Splits into train / validation / test with stratification and your class-balancing settings.
5. Fits a `sklearn.RandomForestClassifier` on the training fold.
6. Evaluates on validation and test folds, reports accuracy and macro-F1, and (when enabled) summarises threshold-sweep behaviour at multiple τ values for τ -selection guidance.
7. Saves the trained model as `<dataset_stem>_rf_model.skops` next to the trainable CSV, with sidecar files for the label encoder (`_labelencoder.joblib`), the feature column order (`_features.json`), and a reproducibility manifest (`training_run.json`) capturing the effective parameters, column order, class list, input file hashes, and library versions.

When training finishes, a confirmation dialog reports test accuracy and macro-F1; the log panel on the main window contains the full per-class metrics. If you provided multiple train-

able CSVs via *Combine Datasets for Training*, the application concatenates them with row-level UIDs preserved before training (useful for tracing which spectrum came from which sample after the fact).

2.8 Run prediction on unlabeled data

Switch to the **Predict** tab.

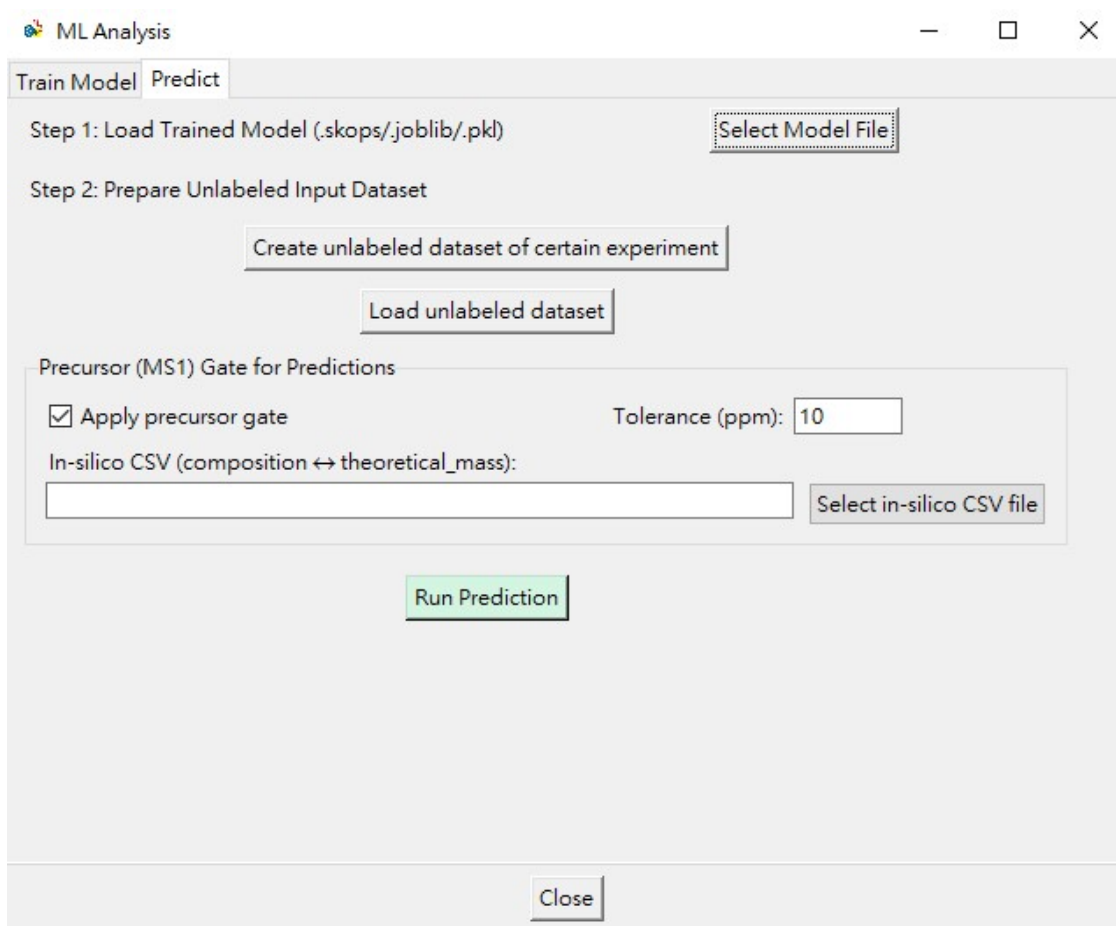


Figure 10. ML Analysis window, Predict tab

Same window, *Predict* tab active. *Step 1: Load Trained Model (.skops/.joblib/.pkl)* with *Select Model File* button. *Step 2: Prepare Unlabeled Input Dataset* with two buttons stacked vertically — *Create unlabeled dataset of certain experiment* (uses an experiment JSON to assemble inputs) and *Load unlabeled dataset* (loads a pre-built unlabeled CSV directly). Below: a labelled panel *Precursor (MS1) Gate for Predictions* containing — *Apply precursor gate* checkbox (enabled by default), *Tolerance (ppm):* entry (default 10), *In-silico CSV (composition ↔ theoretical_mass):* entry + *Select in-silico CSV file* button. Below the gate panel: a green *Run Prediction* button. *Close* button at the bottom.

Step 1 — Model. Click **Select Model File** and pick the .skops file from Step 2.7. The application loads the model, the label encoder, and the feature column list. (For older models saved as .joblib or .pkl, those formats also load; the path of the sidebar files is derived from the model path with the appropriate extension.)

Step 2 — Unlabeled input. You have two options. **Load unlabeled dataset** lets you pick

a pre-built unlabeled CSV directly — useful if you already prepared the input. **Create unlabeled dataset of certain experiment** is the integrated workflow: pick an experiment JSON, and the application traverses each sample’s method JSON, calls `extract_fragment_masses` to build the column order, calls `extract_ion_intensities` to compute features for each spectrum, and writes a `<csv_stem>_unlabeled_ppm{ppm}.csv` next to each sample’s converted CSV. UIDs of the form `{exp_id}:{samp_id}:{scan6digit}` are injected so predictions remain traceable to source spectra.

Precursor gate. The *Apply precursor gate* checkbox (enabled by default) tells the predictor to compare each spectrum’s measured precursor mass against the theoretical mass of the predicted composition. If the deviation exceeds the *Tolerance (ppm)* setting (default 10 ppm), the prediction is moved into the review queue (“greyed out” in the manuscript Figure 4 caption). Provide an in-silico CSV via the *Select in-silico CSV file* button; this file maps composition labels to theoretical masses (the same in-silico CSV from Step 2.4 works).

Run. Click the green **Run Prediction** button. The pipeline:

1. Loads the unlabeled feature matrix and aligns its columns to the model’s training column order. Missing training-feature columns are filled with 0 and extra columns are dropped. (Unlabeled prediction features are encoded at a 20 ppm tolerance — more permissive than the 10 ppm used for training-set encoding; the precursor mass-gate below applies a stricter 10 ppm quality check.)
2. Calls `model.predict_proba` to obtain class probabilities, picks the top class as the prediction, and computes a *confidence* score for that prediction (top class probability).
3. Applies the confidence-threshold demotion: predictions with confidence below τ or with margin (top minus second probability) below *margin* are moved to the *to_review* queue; the rest stay in the main predictions file.
4. Applies the precursor gate (if enabled): predictions whose theoretical precursor mass deviates beyond tolerance are also moved to the review queue.
5. Calls the prediction reporter, which writes `predictions_with_conf.csv`, `class_summary.csv`, `run_summary.json`, `run_summary.md`, and one or more `to_review_*.csv` queues, then pops up a summary dialog showing total spectra predicted, per-class counts, mean confidence per class, and the size of each review queue.

2.9 Read your outputs

Open the output folder named for the prediction run (created next to your unlabeled input CSV). You will find:

Recommended for reviewing results:

- **class_summary.csv** — one row per predicted class, with spectra counts per composition. This is your headline result for profiling.
- **predictions_with_conf.csv** — one row per predicted spectrum (`MS2scan_no`, `predicted_label`, `protonatedmass`, `theoretical_mass`, `ppm_error` when the gate is applied, `source uid`). Use it to trace any individual composition back to its scans.
- **Precursor-gate outputs** (`*_gate_passed.csv`, `*_gate_debug.csv`, `*_PG_counts.csv`) — show which predictions agree with the measured precursor mass; the practical quality filter for composition-level results.
- **run_summary.json / run_summary.md** — reproducibility metadata (model path, input path, settings, version, output inventory) plus per-run statistics.

Not recommended at laboratory scale (`to_review_low_conf.csv`, `to_review_low_margin.csv`, `to_review_ambiguous_topk.csv`): these confidence/margin review queues are a residual feature retained for future use with much larger, well-balanced datasets. At typical per-class spectra counts the predicted-probability and margin values are not well calibrated, so these queues are not informative — and with the confidence threshold disabled by default

they are usually empty or arbitrary. Tuning them meaningfully would require ground-truth labels and hyperparameter-style optimization that small datasets cannot support; we therefore do not recommend interpreting them for routine analysis.

For composition-level profiling, read `class_summary.csv` together with the precursor-gate outputs; for any spectrum that matters individually (e.g., entries you intend to publish), trace it through `predictions_with_conf.csv` and inspect the spectrum itself.

That's the end-to-end CGA pipeline. You started with raw instrument files and ended with predictions on unlabeled spectra plus reviewer-friendly reports.

Part 3 — Workflow guides

This Part covers the workflows that branch off from the Quickstart's CGA spine. Each subsection is independent — read the ones relevant to your data.

Shared across both routes (MAS and CGA). Several steps are route-agnostic: they are configured once in the **Prepare Dataset** window and then applied to whichever labeling route you run. Specifically — **negative sampling** (§3.4; applied during *MAS Merge* and *CGA → Trainable*), **ion mining** (§3.5; applied during *MAS Merge* and *Start CGA analysis*), the **selected-fragment-ion feature encoding** (§4.3), and the **ML training/prediction** stage (§2.7–2.9). In other words, MAS and CGA differ only in how labels are produced; everything downstream is the same pipeline.

3.1 The MAS pipeline with expert annotations

Use the MAS pipeline (Manual annotation support) when you have **expert-validated annotations** linking specific MS2 scans to specific glycan compositions. MAS skips the in-silico library and constraint scoring entirely; instead, it ingests your annotation Excel as ground truth, validates that the labels are consistent with the spectra, and passes everything through the same feature-extraction and ML stages as CGA.

Inputs. Beyond the converted scan CSV and metadata JSON (Step 2.2 of the Quickstart), MAS needs an annotation Excel with two sheets:

- **MSlist sheet** — one row per annotated scan. Required columns include the scan identifier and the assigned glycan composition label. Additional metadata columns (RT, intensity, manual notes) are preserved but not consumed.
- **ionlist sheet** — reference fragment ions for feature extraction. Must contain a mass column (recognised header variants: `mass`, `mz`, `ion_mz`, `m/z`, `fragmentation_mass`). The `fragmentation_mass` alias was added to support Score B workbooks acting as ion lists in the MAS path; it is renamed to `mass` internally.

Walkthrough. From the Prepare Dataset window with your scan CSV(s) added per Quickstart Step 2.3:

1. Click **Add Excels (ion list / man annotation)** and select your annotation Excel. The application appends the file to the corresponding sample's excel slot.
2. Make sure each sample has its metadata attached (visible in the tree as a *Metadata: ...* sub-row). If not, right-click the sample and choose **Open Metadata Editor**.
3. Right-click the sample and choose **Link and Validate Sample** (or click the **Link Sample** button in the action grid with the sample selected). The application runs six fail-closed validation stages: sample lookup, file presence, CSV structure, Excel sheet existence, Excel content (validates the MSlist sheet structure and ensures the ionlist sheet has a

numeric mass column), and metadata gate. On success, the sample's tree status changes to "OK" and a per-sample method JSON of family MAS is written. On any failure, an error dialog names the offending stage and the sample's tree status changes to "warning" or "error".

4. With the sample now validated, click **Merge Sample**. The button is greyed out until validation succeeds. The application asks you for an output folder, reads Derivatization Type from the metadata (and prompts you for a custom mass shift if the value is Others), then builds a merged dataset by joining the converted CSV's spectra with the annotation Excel's labels.
5. If *Add Non-glycan entries* is enabled in **Negative Sampling Setting** (Section 3.4), low-evidence scans not annotated in the Excel are sampled as Non-glycan negatives at this stage.
6. The merged CSV is written to your chosen folder as {sample_name}_merged_{timestamp}.csv. The path is recorded in the sample's files["trainable_csv"] slot, and a method JSON of family MAS is refreshed.

Output schema. The merged CSV has the same columns as a CGA-pipeline trainable CSV: MS2scan_no, the per-fragment ion-mass feature columns, and Structure (the expert-assigned label, or Non-glycan for sampled negatives). Train and predict from this point onward exactly as in Quickstart Steps 2.7–2.8.

Differences worth noting compared with CGA. MAS labels are not scored — the application trusts the expert. The application *does* validate that Excel-listed scans exist in the converted CSV and that the ion-list mass column is well-typed, but it does not double-check the assigned compositions. Score B is not applicable to MAS rows because Score B operates on candidate compositions; MAS rows have a single expert-assigned composition with no candidate set.

3.2 Combining trainable datasets (and when not to)

GlycoMSP can combine multiple trainable CSVs into one training pool via **Combine Datasets for Training** in the ML Analysis window's Train tab. Click **Select Datasets**, pick the CSVs (Cmd-/Ctrl-click for multi-select), and the application concatenates them with per-row UIDs preserved for traceability, loading the result into *Step 1*.

The one hard requirement: the same fragment-ion list. Trainable datasets are only safely mergeable when they were built from the **same fragment-ion (feature) list**, so that their feature columns mean the same thing. Combining datasets built from different ion lists produces a misaligned feature space and the model training will not identify features correctly (or will fail). This is the only compatibility GlycoMSP can guarantee at the file level.

Scientific compatibility GlycoMSP does *not* enforce — your responsibility. Even with a shared ion list, only merge datasets acquired under compatible conditions:

- **Same ionization mode** (positive vs. negative) and **same derivatization** (do not mix native and permethylated glycans — they fragment differently). Mixing these is improper and, because the feature ions differ, will generally fail at training unless you deliberately prepared a single ion list valid for both and built every dataset with it.
- **Different biological treatments** of the same system are fine to group.
- For building a comprehensive glycan database, even **different cell types or species** may be pooled — *provided the sample-preparation protocol and the spectral-acquisition workflow are identical*.

Mixing CGA-derived with MAS-derived rows is generally not recommended. CGA labels are simulated/constraint-scored pseudolabels and are less discriminative than expert MAS labels; mixing the two typically *reduces* performance rather than helping. In addition, if the MAS data uses customized (non-composition-string) labels, combining it with CGA output

— which uses composition strings — leads to inconsistent label spaces and training errors. Mix the two routes only if you have a specific reason and have confirmed label and feature compatibility. The **CGA dataset?** checkbox in *Step 1* only records the labeling origin in the run metadata; it does not change training behavior or reconcile any of the above.

3.3 Score B enrichment — motif-aware re-ranking

Score B is a second-stage scoring pass that re-ranks CGA candidates using user-defined glycotopic motif rules. The manuscript Methods describes Score B as a *motif-aware re-ranking step that refines glycan composition assignments generated by CGA*. Use it when your samples include compositions that are isobaric (identical in mass, so Score A cannot discriminate them) but distinguishable in fragmentation pattern. For example, in permethylated glycans a Neu5Gc→Neu5Ac change ($-\text{CH}_2\text{O}$) combined with a Fuc→Hex change ($+\text{CH}_2\text{O}$) cancels out, so a ...G...F...-containing composition is mass-degenerate with a ...S...H... one — same precursor mass, different composition.

By default Score B is **inactive**. Enabling it requires a Score B workbook authored externally. The manuscript (Supplementary Data 2 in manuscript/GlycoMSPdraft/) provides example workbooks and authoring guidance; this manual covers only how to attach and consume the workbook.

Workbook structure. The Score B workbook is a six-sheet Excel file (.xlsx, .xlsm, .xltx, or .xltm). The sheets encode, in order: glycotope motif definitions, direct-evidence fragment rules, gated-evidence rules (rules that fire only when a precondition rule has fired), composition-consistency rules (penalties for evidence inconsistent with the candidate composition), unexpected-evidence rules (penalties for evidence that does not fit any expected motif), and a configuration sheet. The exact sheet names and column layouts are documented in `score_b_config_instructions.md`.

Attaching the workbook. From the CGA Setup window (Quickstart Step 2.4):

1. Click **Add Score B Workbook** and pick the Excel file. The path appears in the *Score B workbook* field.
2. Click **Validate Score B Workbook**. The application runs a load test against `msp_CGA_structscore.ScoreBLoader` and pops up either a confirmation dialog (“Workbook validated”) or an error dialog naming the offending sheet. Validation is non-destructive — it only reads.
3. Optionally, click **Clear Score B Workbook** to detach. Detaching is the same as having never attached.

Then proceed with **Start CGA analysis** as in Quickstart Step 2.5.

Output columns when Score B is active. The pseudolabel TSV gains four columns:

- **score_b** — float in $[0, 1]$, the final Score B value for this candidate.
- **score_b_rank** — per-scan rank by Score B (1 = highest).
- **score_b_selected** — boolean, true only for the top-ranked candidate per scan that has a non-empty `selected_composition`.
- **score_b_motif_summary** — short text summary of which motifs contributed and which penalties applied.
- **selected_composition** — the chosen composition string for selected rows; empty otherwise.

Plus the formula behind `score_b`:

$$b = \max(0, \min(1, \text{support} - \text{penalty_comp} - \text{penalty_unexpected}))$$

where *support* is motif-level evidence aggregated across the workbook’s glycotopes, *penalty_comp* penalises evidence inconsistent with the candidate’s composition, and *penalty_unex-*

pected penalises evidence not accounted for by any expected motif. The clamp ensures all candidates remain on a comparable $[0, 1]$ scale.

Selection rule in CGA → Trainable. When you build the trainable CSV (Quickstart Step 2.6), the application detects the presence of `score_b_selected` and switches selection logic: per scan, the row with `score_b_selected = true` is taken as the assigned label. For scans where no candidate was Score B-selected (typically because no motif evidence was strong enough), the workflow reverts to Score A selection — the candidate with highest `ion_score` above the minimum threshold. The manuscript names this “*Score B preferred, Score A fallback*”.

Failure mode. Score B execution is non-fatal. If `enrich_cga_tsv_with_score_b` raises any exception (workbook-load error, malformed rule, runtime issue), the application logs a warning, shows a non-blocking message, and returns the original Score-A-only TSV unchanged. Subsequent CGA → Trainable runs against that TSV simply use Score A selection. You will not lose your CGA work to a Score B failure.

When motif-aware re-ranking helps. Score B improves discrimination for isobaric same-mass candidates, particularly compositions that share monosaccharide counts but differ in glyco-feature expression (e.g., Lewis antigens versus their sialylated counterparts). It does not help when no motif evidence exists (the workbook rules don’t match any peaks for the spectra in question), in which case the Score A fallback applies and Score B is effectively a no-op.

3.4 Negative sampling options

Both the MAS merge and the CGA → Trainable pipelines can sample low-evidence scans as Non-glycan negatives, providing the ML model with examples of the “not a glycan” class. The configuration is shared via a single dialog: **Negative Sampling Setting** in the Prepare Dataset window’s action grid.

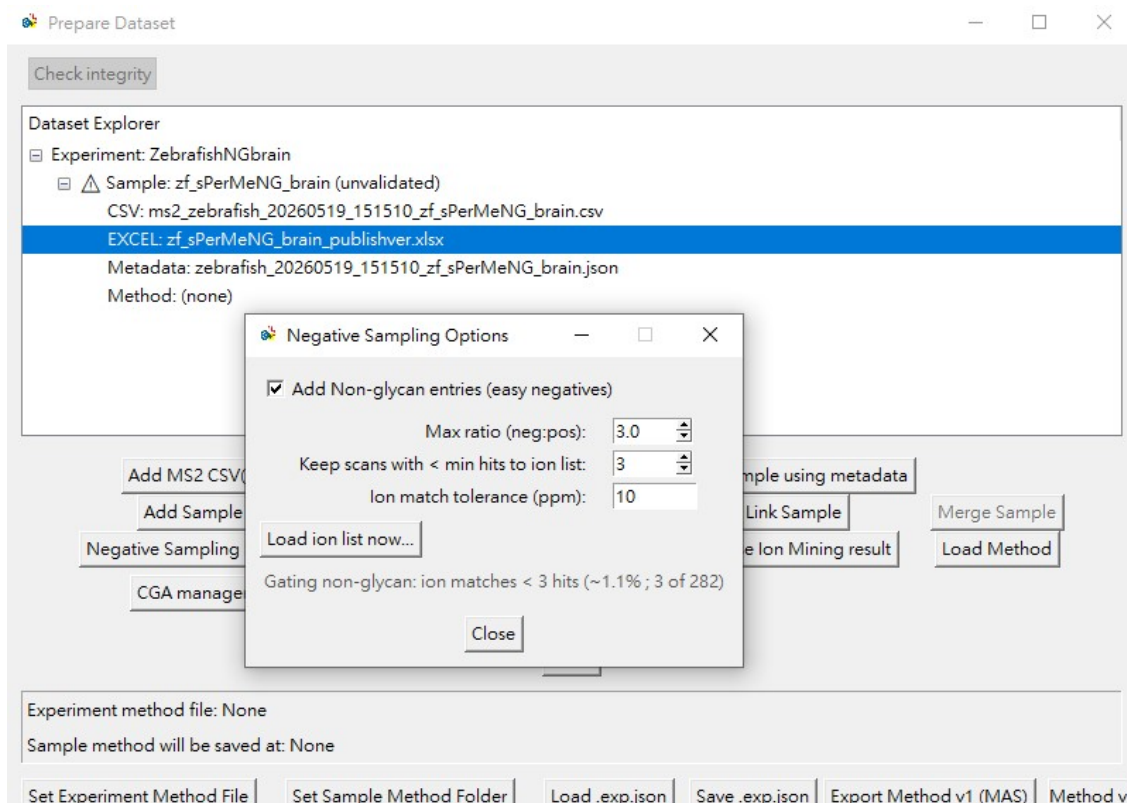


Figure 11. Negative Sampling Setting dialog

Modal dialog. *Add Non-glycan entries (easy negatives)* checkbox. Below: *Negative-to-positive ratio* Spinbox (range 0.0–10.0, step 0.5; **default 3.0**). *Min ion-hit gate* Spinbox (range 0–50, step 1; **default 3**). *PPM tolerance* Entry with input validation (**default 10**). Below: live hint label reading “*Gating non-glycan: ion matches < K hits (~P% ; K of N)*” where K is the gate value, P is the percentage of total features, and N is the auto-probed feature count from the current sample’s ion list. *Load ion list now...* button (manually re-probes the feature count). *Close* button.

The dialog **does not run any workflow itself** — it only sets shared state. The next time you run **Merge Sample** (MAS), **Start CGA analysis** (which populates the pseudolabel TSV with optional pre-mining negatives), or **Build Trainable CSV** (CGA → Trainable), the consumer reads these values.

What the gate does. A scan qualifies as a Non-glycan negative only if its ion-hit count against the reference ion list is **below** the *Min ion-hit gate*. So if you set the gate to 3, scans matching 0–2 ion-list fragments within the PPM tolerance are eligible negatives; scans matching 3 or more are skipped. The ratio control then caps the total negatives sampled at $ratio \times n_{positives}$. Default settings (**ratio 3.0, gate 3, ppm 10**) cap negatives at three times the number of positives, sampled from non-positive scans matching fewer than 3 ion-list fragments.

Recommended starting point. Start with the defaults. If your model overfits to the positive class (test recall on Non-glycan is poor), raise the ratio to 2.0 or 3.0 to give the model more negative examples. If your negatives leak into the positive class (the model predicts Non-glycan for clear glycan spectra), raise the gate so only truly low-evidence scans qualify as negatives.

Cross-pipeline state. The dialog’s settings affect MAS merge, CGA → Trainable, and the optional pre-mining negatives in CGA analysis. Setting them once and leaving them is fine.

3.5 Ion mining — discovering diagnostic fragments

Ion Mining inspects your data for fragment masses that distinguish glycan-class spectra from negative spectra and would not be in a generic ion list. The output is a ranked list of suggested m/z values you can review and optionally add to your ion list for future runs.

Open the configuration via **Ion Mining** in the Prepare Dataset action grid.

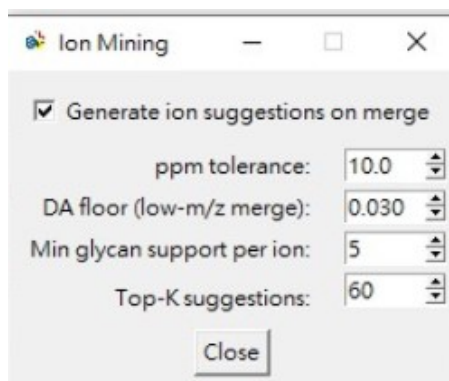


Figure 12. Ion Mining dialog

Modal dialog, 5-row grid. *Generate ion suggestions on merge / on CGA → Trainable* checkbox. Below: *PPM tolerance* Spinbox (1.0–50.0, step 0.5; default 10.0). *Da floor for low-m/z merging* Spinbox (0.00–0.10, step 0.005, format %.3f; default

0.030). *Min support per glycan class* Spinbox (1-100; default 5). *Top-K suggestions exported* Spinbox (5-500, step 5; default 60). *Close* button.

Like the Negative Sampling dialog, this one only sets shared state. The actual mining runs during dataset building for **either** route when the enable checkbox is ticked — specifically during **MAS Merge** and during **Start CGA analysis** (it is not part of the CGA → Trainable step).

How mining works at a high level. The application takes your current spectra and assigns each one a binary label (Glycan for annotated/positive scans, Non-glycan for sampled negatives — the same negatives configured in Section 3.4). It then groups m/z peaks across spectra at the configured PPM tolerance with the small-mass merge floor (so, e.g., peaks at 204.085 and 204.087 collapse to one cluster), counts how many Glycan vs Non-glycan spectra contain each cluster, and computes statistics: support count per class, lift (ratio of class support to baseline support), odds ratio, and a “recommended” flag for clusters meeting promotion thresholds. The top-K clusters by lift are exported.

Reviewing results. Click **Browse Ion Mining result** in the action grid. The application opens a modal viewer showing the top 10 suggestions in a sortable Treeview with columns `mz`, `support_glycan`, `support_non`, `lift`, `odds_ratio`, `already_in_list`, `recommended`. Sort priority is lift (descending), then support count. Click a row and use **Copy selected m/z** to copy the value to clipboard for adding to your ion list. **Open folder** opens the OS file manager at the suggestions CSV’s parent directory so you can audit the full list outside the GUI.

Output file. Each enabled mining run writes `{sample_name}_ion_suggestions.csv` next to the merged or trainable CSV. The viewer reads from a closure-scoped path variable (`last_suggest_csv_var`), so it always shows the most recent run; you can re-load older suggestion CSVs by clicking the viewer’s open dialog and picking the file directly.

When to use ion mining. Two natural cases. First, **early ion-list curation** — if you started with a generic ion list and want to expand it with class-specific diagnostic fragments observed in your own data. Second, **method troubleshooting** — if your model performs poorly, mining can reveal fragments your reference list missed but that consistently distinguish glycans in your dataset. In both cases, the suggestions are not added to the ion list automatically; you decide which to incorporate, edit the ion-list Excel, and re-run.

3.6 Building unlabeled datasets across multi-sample experiments

The Predict tab has two ways to load an unlabeled input. **Load unlabeled dataset** picks a single pre-built CSV and is what the Quickstart used. **Create unlabeled dataset of certain experiment** is the integrated multi-sample workflow.

When to use it. When you have an experiment containing several samples and want to predict on all of them in one go, with consistent feature spaces and traceable outputs. This is the typical setup for, e.g., a clinical cohort: the experiment is the cohort, the samples are individual patients, and you want one trained model applied uniformly.

Walkthrough. With a trained model selected (Step 1), click **Create unlabeled dataset of certain experiment** and pick the experiment JSON (`*.exp.json`) for the cohort. The application:

1. Reads the experiment JSON and traverses the samples array.
2. For each sample, locates the per-sample method JSON via the sample’s methods array, reads its `inputs.converted_csv.path` and `inputs.ion_list.path`, and writes back any missing path in the experiment JSON if it can resolve them at runtime.
3. Calls `extract_fragment_masses` to read the ion list and determine the column order, then `extract_ion_intensities` to compute the feature matrix for each spectrum (PPM-windowed, log10-transformed, identical to training-time feature extraction).

4. Injects per-row UUIDs of the form `{exp_id}:{sample_id}:{scan_six_digit_no}` so each predicted row is traceable to source experiment, source sample, and source scan number.
5. Writes one `{csv_stem}_unlabeled_ppm{ppm}.csv` per sample, next to that sample's converted CSV.
6. Persists the unlabeled-CSV paths back into the experiment JSON's per-sample blocks (so re-running picks up the cached paths instead of recomputing).

After this completes, click **Run Prediction**. The prediction step consumes each unlabeled CSV in turn and writes its output folder next to the input.

A note on consistency. The unlabeled feature space *must* align with the trained model's feature space. The application enforces this by reordering input columns to match the model's `train_feats` and filling any missing column with 1.0 (the no-peak default), with extra columns dropped. If your experiment uses a different ion list from the one used at training time, predictions will work but may be noisier — every “missing” reference fragment becomes an effective zero. For best results, use the same ion list (or a strict superset) at training and prediction time.

Part 4 — Concepts

This Part explains the design choices behind GlycoMSP. Reading is optional for a first run, but useful for thesis review and for understanding why a particular pipeline behaves the way it does. Formulas and naming follow the manuscript Methods.

4.1 Score A — ion-fitting score

Score A (manuscript variable *a*) is the **ion-fitting score**: the proportion of diagnostic ions detected in an MS2 spectrum, evaluated against the user-defined ion list. Per scan and per candidate composition, the application:

1. Takes the candidate's set of expected reference ions from the ion list.
2. For each expected ion, searches the spectrum's peak list for a peak within the configured PPM tolerance (default 10 ppm for ion-list matching).
3. Optionally applies an “anchor” gate — a minimum number of high-intensity required reference ions that must be detected for the score to be computed at all. When the gate fails, the score is zero (or, in fallback mode, a simple fraction).
4. Computes the score as `hit_count / total_reference_count`, normalised to `[0, 1]`.

The manuscript frames Score A as serving “as a general filter to reject spectra having possible compositions without enough glycan-related fragments.” It works at the **fragment-evidence level** — does the candidate composition predict fragments that we actually see? — without considering structural detail beyond fragment presence.

In practice, Score A is what drives candidate selection in CGA when Score B is inactive. For each scan, the candidate with the highest Score A above the user-set minimum threshold is taken as the assigned label. The threshold is exposed in the **CGA → Trainable** modal (Quickstart Step 2.6). Score A is a ratio in `[0, 1]` (matched ions ÷ total ions in the list); GlycoMSP applies a default minimum of 0.07. The manuscript does not prescribe a threshold value beyond this default — the choice is left to the user.

If the per-scan score attach fails, GlycoMSP falls back to a simpler scorer (`_fallback_simple_ion_scoring`) that matches the **same user-provided ion list** against each spectrum and computes the matched-fraction score. (Earlier builds used a small set of hardcoded anchor

masses here; that was removed during the v14 scoring-logic fix, so the fallback now always scores against your configured ion list.) A properly configured ion list matching your derivatization chemistry is therefore required — there is no built-in default ion set.

4.2 Score B — motif-aware re-ranking

Score B (manuscript variable *b*) operates one level above Score A, on **structural motif evidence**. Where Score A asks “*do the expected fragments appear?*”, Score B asks “*do the appearing fragments imply the structural motifs that this candidate would predict?*”. The manuscript defines:

$$b = \max(0, \min(1, \text{support} - \text{penalty_comp} - \text{penalty_unexpected}))$$

where:

- **support** quantifies motif-level evidence assembled from user-defined glycotopic rules. Each rule contributes positively when its diagnostic fragments appear (subject to gate conditions if the rule is gated); contributions sum across all rules in the workbook.
- **penalty_comp** subtracts evidence that contradicts the candidate’s composition — for example, a sialic-acid-specific fragment in a candidate that has no sialic acid.
- **penalty_unexpected** subtracts evidence that no expected motif accounts for — peaks that look diagnostic but match no rule applicable to this candidate.
- The outer $\max(0, \min(1, \dots))$ clamps the result to the unit interval, so all candidates remain on a comparable $[0, 1]$ scale regardless of motif-rule density.

The selection rule used downstream in CGA → Trainable is what the manuscript names “*Score B preferred, Score A fallback*”: the per-scan top-ranked Score B candidate is selected when it has a non-empty assigned composition; otherwise the workflow reverts to Score A’s top-ranked candidate. In practice this means Score B refines decisions where motif evidence discriminates, and stays out of the way where it doesn’t.

Score B is **inactive by default**. Activation requires a Score B Excel workbook authored by someone with glycotopic domain expertise; see Section 3.3 for attaching the workbook and the manuscript’s Supplementary Data 2 for authoring instructions. The manuscript explicitly recommends that users without established glycotopic knowledge proceed with Score A alone (CGA-ML) and validate predictions manually rather than attempt Score B configuration without the prerequisite background.

4.3 Feature extraction

Both pipelines (CGA and MAS) produce feature matrices using the same per-cell formula. For each spectrum and each reference ion *n*:

$$i_{\text{feature},n} = \log_{10}(I_n) + 1$$

where *I_n* is the maximum peak intensity within the configured ion-matching tolerance of fragment *n*’s reference mass. When no peak matches, the cell takes the default value 1.0. Training-set feature encoding uses a **10 ppm** tolerance (both MAS-merge and CGA→trainable); unlabeled **prediction** features are encoded more permissively at **20 ppm**, with a stricter **10 ppm** precursor mass-gate applied as a quality check at prediction time.

Two design choices worth understanding. First, **the log transform is applied before the +1, not after**: the formula is $\log_{10}(I) + 1$, not $\log_{10}(I + 1)$. The two are equivalent only at high intensity; at low intensity they differ. The application’s choice means that a peak with raw intensity 1.0 produces a feature value of 1.0 (clean coincidence with the no-hit default), peaks below 1.0 produce values less than 1.0 (so weaker hits are distinguishable from no-hits but compressed), and peaks above 1.0 produce values greater than 1.0 (the dominant signal

range). Second, **no-hit cells default to 1.0 rather than 0.0**: this lets the Random Forest treat “no peak in window” as a distinct, low-but-nonzero baseline rather than collapsing it into the same bin as a tiny hit, which would distort feature-importance estimates.

Feature extraction is identical for CGA-derived and MAS-derived rows. The model trained on a mix sees one feature space and treats all rows uniformly.

4.4 The ML model — Random Forest

GlycoMSP ships a single, well-tuned Random Forest classifier. The choice was deliberate: Random Forests handle mixed feature scales without normalisation (intensities span several orders of magnitude even after log-transform), are robust to small datasets and class imbalance, expose interpretable feature importances (useful when explaining model behavior to chemists), and are fast enough to train and predict on commodity laptops. The frozen v1.10 configuration is 400 decision trees, `max_depth=None` (unrestricted depth), `min_samples_split=2`, `min_samples_leaf=1`, `class_weight="balanced"`, and `random_state=42`; scikit-learn defaults apply for the rest (e.g. `max_features="sqrt"` and bootstrap sampling). These are the values reported in the manuscript.

Threshold τ + margin. The application supports a confidence-demotion step at prediction time, **disabled by default** in v1.10. The top-class probability for each prediction is compared to a threshold τ (default 0.65). When the top probability is below τ , OR when the margin (top minus second probability) is below a separate threshold (default 0.05), the prediction is *demoted* — sent to a review queue rather than included in the main predictions. The two thresholds together protect against two distinct failure modes: τ catches “the model is unsure overall” cases, and margin catches “two classes are nearly tied” cases.

The threshold step is disabled by default, which is the recommended setting for small datasets (with the checkbox off, the application uses permissive values $\tau=0.0$, `margin=1.0` — i.e., everything kept). Threshold demotion mostly helps once datasets exceed a few hundred spectra per class, where probability calibration becomes meaningful.

Class balancing. Both Score-A-only and Score-B-active runs can produce class-imbalanced datasets (more candidates for common compositions, fewer for rare ones; lots of Non-glycan negatives sampled at default ratios). The application includes two mitigations: per-class minimum size (drop classes with fewer than 5 spectra; manuscript baseline) and majority-class capping (cap the dominant class — typically Non-glycan — at no more than 3× the largest glycan class; manuscript baseline). Both are configurable in the Train/Test Parameters dialog. The defaults reflect what worked across the development datasets and are reasonable starting points.

4.5 Reproducibility — `training_run.json`

Each successful training run writes a JSON manifest next to the model file. The manifest captures everything needed to reproduce the run given the same input data:

- **effective_params** — the fully merged ML parameters that were actually used (UI vars + JSON editor + presets layered together).
- **column_order** — the exact ordered list of feature columns. Critical: at predict time the application reorders unlabeled-input columns to match this list; without it, Random Forest predictions become positional gibberish.
- **classes** — the ordered list of class labels seen during training.
- **inputs** — the trainable CSV path(s) and SHA-256 hashes of each.
- **hashes** — same as inputs; redundancy is intentional.
- **versions** — environment versions: Python, platform, sklearn, joblib, numpy, pandas, and the GlycoMSP build version.

- **created_at** — local-time ISO timestamp of when training completed.

This manifest accompanies the model file (*.skops plus *_labelencoder.joblib plus *_features.json). When you publish a model or share it with a collaborator, include all four files. The manifest is the document that lets reviewers verify a run was performed under the parameters reported in a paper.

4.6 Why two parallel labeling pipelines?

The architecture sits on a deliberate observation: the bottleneck for ML in glycomics is not the classifier — it’s the labels. Most public glycomics datasets are not annotated in a way that supports cross-study ML, and even within a single lab the cost of expert annotation makes it impractical to manually label every spectrum.

GlycoMSP addresses this with two complementary entry points. **CGA** generates labels computationally from in-silico libraries and constraint scoring. The labels are imperfect by construction — Score A is a heuristic, Score B helps but requires expert workbook authoring — but they are *available*, in volume, for any dataset where you can specify the chemistry. **MAS** ingests expert-validated labels when you have them. The labels carry information that no rule-based system can match (subtle structural assignments, ambiguity resolution from human inspection), but they require expert time and tend to exist only for studies that have been carefully reviewed.

The design choice to share **feature extraction and ML stages** across both pipelines is what makes mixing possible. A researcher starting a new line of work might collect a small expert-annotated set (MAS rows) and combine it with a larger CGA-derived set on the same instrument, training a single classifier on the union. The expert labels anchor the model on confident assignments; the constraint labels expand its exposure. This is the *CGA-ML / MAS-ML* duality the manuscript describes — two ways into the same trained-model surface.

4.7 Ion mining (terminology)

The Prepare Dataset window has a button labelled **Ion Mining** (and a sister button **Browse Ion Mining result**). This manual uses “ion mining” throughout, matching the GUI label. The procedure is a frequency-and-lift analysis that ranks m/z peaks by how preferentially they appear in glycan vs Non-glycan spectra, with Laplace add-one smoothing to handle sparse counts.

Conceptually the same procedure could be called *feature mining*, since it surfaces candidate features for the ML model — but that name is not used in the manuscript and is not adopted as a formal term here. The GUI term “**ion mining**” is authoritative throughout this manual.

4.8 Evaluating MAS-ML: within-tissue, leave-one-tissue-out, and pooled CV

Because GlycoMSP trains *local* models, “performance” depends on which spectra the model has seen. The manuscript evaluates MAS-ML three complementary ways, and understanding the distinction matters when interpreting any GlycoMSP model:

- **Within-tissue held-out test** — split one tissue’s MAS data into train/test (80/20, stratified, classes with ≥ 5 spectra). This asks: *can the model recover held-out scan-level labels from the same sample context?* It is the cleanest in-sample check and the strongest numbers (zebrafish glycan macro-F1 ≈ 0.93 -0.95).
- **Leave-one-tissue-out (LOTO)** — train on two tissues, predict the third (held-out) tissue as an independent biological sample. Because tissues do not share identical glycan

profiles, held-out compositions are split into **seen** classes (present in the training label space) and **out-of-label-space** classes (absent from training). Metrics are reported on seen classes only; out-of-label-space compositions are reported separately, **not** as classification errors (a supervised model cannot predict a label it never saw). LOTO performance is lower than within-tissue (≈ 0.60 – 0.83), which is expected and reflects label-space mismatch, not model failure.

- **Pooled cross-validation** — pool all tissues and run stratified 5-fold CV. Pooling raises the number of trainable compositions (≈ 14 – 25 per tissue $\rightarrow 43$ pooled) and stabilises performance (macro-F1 $\approx 0.90 \pm 0.04$).

The practical lesson: a local RF model should be used **within its trained label space**, and accumulating compatible, manually reviewed spectra (pooling) is how you expand coverage. Primary metrics are always glycan-only — the operational Non-glycan class (an *estimated* negative, not ground truth) is reported separately. Part 8.2 gives the exact commands and expected numbers; the standalone reproduction script and its tech note document the metric definitions in full.

Part 5 — Reference

This Part is the reference. It is denser than Parts 1–4 and intended to be searched, not read linearly. Schemas, default values, and output column specifications are listed exhaustively where useful for reproducibility.

5.1 Input file formats and column requirements

Scan CSV (output of Step 2.2, input to everything else)

Produced by GlycoMSP's converter (`msexp.convert_raw_to_csv` for RAW, `msexp.extract_mzML` for mzML). One row per MS2 scan.

Column	Type	Notes
MS2scan_no	int	Scan number from the instrument file. Primary key for downstream joins.
protonatedmass	float	Precursor mass in monoisotopic protonated form. Used by the precursor MS1 gate.
peaklist	str	Semicolon-separated m/z list of peaks in the spectrum.
peakintensity	str	Semicolon-separated intensity list (same length as peaklist).
MS1scan_no	int	Parent MS1 scan number.
MS1_isolationmass	float	MS1 isolation (precursor) m/z.
MS1_monoisolationmass	float	MS1 monoisotopic isolation m/z.
chargeState	int	Precursor charge state.

Retention time (RT) and other LC/EIC-related parameters are **not** retained — GlycoMSP is built around MS2 scan-level traceability tied to the peak list and intensities, not chromatographic features.

The metadata sidecar `<csv_basename>.json` (alongside the scan CSV) records Experiment Title, Glycan Type, Mass Analyzer charge mode, Derivatization Type, Reduced, Sample Type, Operator, Original raw file path, Raw filename, plus `json_type = "glycomsp.metadata"` and `schema_version = "1.0.0"` headers.

Annotation Excel (MAS pipeline)

The expert-prepared annotation file consumed by the MAS pipeline. The manuscript Methods names the four sheets explicitly:

- **GlycanList** — annotator’s working summary of glycan compositions observed. Not consumed by GlycoMSP.
- **MSList** — manually assigned glycan compositions paired with MS2 scan numbers. Required columns: scan number, glycan composition. Optional columns: peak m/z, charge state, mass shift, adduct type, free-form notes. Consumed by `directassign_files`.
- **Ionlist** — fragmentation ions used by GlycoMSP for feature extraction. Required column: a numeric mass column. Recognised header variants: `mass`, `mz`, `ion_mz`, `m/z`, `fragmentation_mass` (the last is for compatibility with Score B workbooks acting as ion lists).
- **Information_Sheet** — operator metadata. Not consumed by GlycoMSP.

A template is included in the GlycoMSP project folder.

Ion list (CGA pipeline, also acceptable as MAS Ionlist sheet)

Single-sheet CSV or Excel. One row per reference fragment.

Column	Required	Recognised header variants	Purpose
Mass	Yes	<code>mass</code> , <code>mz</code> , <code>ion_mz</code> , <code>m/z</code> , <code>fragmentation_mass</code>	Reference m/z for PPM matching
Identifier	No	<code>structural_identifier</code> , <code>glycotope_name</code> , <code>glycotope</code> , <code>fragment_name</code> , <code>fragment</code> , <code>ion_name</code> , <code>annotation</code> , <code>name</code> , <code>label</code> (in priority order)	Human-readable label; used in spectrum-preview annotation and ion-mining reports

Additional columns are preserved but not consumed.

In-silico glycan library (CGA pipeline)

Generated by **Generate In-Silico CSV** in the CGA Setup window, or supplied externally. CSV with columns `Mass`, `Hex`, `HexNAc`, `NeuAc`, `NeuGc`, `KDN`, `Fuc`, plus (in negative-mode runs) `HexA`, `S03`, `P03H`. One row per theoretical composition consistent with the user-supplied flag set. Reusable across samples sharing derivatization and ionization mode.

Score B workbook (optional, CGA pipeline)

Six-sheet Excel file (`.xlsx`, `.xlsm`, `.xltx`, or `.xltm`). See Section 5.6 for the high-level sheet structure; full authoring guidance is in `score_b_config_instructions.md` (manuscript supplementary).

5.2 Method JSON v1 schema

Produced by `build_method_v1_from_tree`; written by `save_method_v1_for_sample` and `_export_method_v1`. One file per (sample, family) pair.

```
{
  "json_type": "glycomsp.method",
  "schema_version": "1.0.0",
```

```

"uid": "<uuid4>",
"created_utc": "<iso8601>",
"updated_utc": "<iso8601>",

"method": {
  "family": "MAS" | "CGA",
  "name": "<exp_name>:<sample_name>:<family>",
  "description": "",
  "tags": []
},

"sample": {
  "sample_name": "<sample_name>",
  "experiment_title": "<exp_title>"
},

"inputs": {
  "converted_csv": {"path": "<native-normpath>"},
  "metadata_json": {"path": "<native-normpath>"},
  // MAS only:
  "annotation_excel": {"path": "<native-normpath>"},
  // CGA only:
  "ion_list": {"path": "<native-normpath>"},
  "insilico_glycan_list": {"path": "<native-normpath>"},
  // CGA optional:
  "score_b_workbook": {"path": "<native-normpath>"
},

"parameters": {
  "mas": {},
  "cga": {},
  "scoring": {},
  "ml": {}
},

"artifacts": {
  "reports": [],
  // CGA only – populated when corresponding sample fields are set:
  "CGAresult_tsv": {"path": "<native-normpath>"},
  "trainable_csv": {"path": "<native-normpath>"},
  "unlabeled_csv": {"path": "<native-normpath>"
},

"validation": {
  "status": "unknown",
  "checked_utc": null,
  "items": []
},

"software": {
  "glycomsp": {"version": "", "commit": ""},
  "extractor": {"name": "", "version": ""}
},

```

```

"operator": {
  "name": "",
  "note": ""
}
}

```

Key contracts:

- method.family determines which inputs keys are populated. MAS reads annotation_excel; CGA reads ion_list + insilico_glycan_list. Both read converted_csv + meta-data_json. score_b_workbook is optional and CGA-only.
- Path values are stored as **native** OS-specific paths via os.path.normpath. Windows produces C:\path\file; macOS / Linux produce /path/file. Cross-platform load is handled at read-time by _resolve_path (Windows-drive regex detection).
- Hash fields are **not currently written** by the writer; the schema reserves them for a future addition.
- artifacts.CGAResult_tsv was previously named pseudolabels_tsv; reads accept either key for backward compatibility, but writes always use the new name.
- The parameters sub-objects are placeholder shells in the current schema; they are reserved for future per-method parameter snapshots.

5.3 Experiment JSON schema

Produced by export_experiment_json; one file per experiment. Holds the experiment's organisational state — sample list, per-sample method references, ML defaults, and policies.

```

{
  "json_type": "experiment",
  "schema_version": "1.x",
  "uid": "<uuid4>",
  "created_utc": "<iso8601>",
  "updated_utc": "<iso8601>",

  "experiment": {
    "title": "<exp_title>",
    "description": "",
    "tags": []
  },

  "workspace": {
    "preferred_method_folder": "",
    "preferred_exp_path": "",
    "ui_order": ["<sample_name>", ...]
  },

  "samples": [
    {
      "sample_name": "<sample_name>",
      "validated": true | false,
      "methods": [
        {
          "method_id": "<uuid4>",
          "method_name": "<basename>",
          "family": "MAS" | "CGA" | "UNKNOWN",
          "method_ref": {

```

```

        "path": "<posix-str>",
        "hash": {"alg": "sha256", "value": "<hex>"},
        "last_seen_utc": "<iso8601>"
    },
    "validation": {
        "status": "ok" | "warn" | "missing" | "incomplete" | "unknown",
        "checked_utc": "<iso8601>",
        "notes": "<str>"
    }
},
...
],
...
},
...
],

"validation_policy": {
    "hash_alg": "sha256",
    "allow_missing_method": true,
    "allow_hash_mismatch": true
},

"ml_defaults": {
    // optional; written/read by the ML params editor's
    // "Apply to exp" / "Pull from exp" buttons
},

"train_parameters": {
    // optional; written by train_model after a successful training run
}
}

```

Key contracts:

- Path values inside `method_ref.path` are stored as **POSIX** strings via `pathcanon.to_posix_str`, regardless of host OS. This keeps experiment JSON files diff-clean across hosts.
- `validation_policy.allow_hash_mismatch: true` is the default — a hash mismatch on import sets the method's status to "warn" but loads the method anyway. Set to false to require exact hash match.
- `ml_defaults` and `train_parameters` are optional top-level keys populated by the ML Analysis window. Their internal structure follows the merged ML-params dictionary schema and is not validated by GlycoMSP beyond JSON parseability.
- `linked_validated_samples` (in-memory) round-trips via the `per-sample validated: true/false` flag. Renaming a sample post-validation without re-saving the exp JSON loses the flag.

5.4 Output file specifications

Pseudolabel TSV (CGA, output of `run_pseudolabeling`)

Tab-separated. One row per (scan, candidate composition) pair.

Column	When present	Notes
MS2scan_no	always	From scan CSV
protonatedmass	always	Joined from scan CSV
comp_str	always	Compact composition label, e.g. F1H5N4
comp_tuple	always	(Hex, HexNAC, NeuAc, NeuGc, KDN, Fuc) ordered tuple
ppm_error	always	Precursor PPM error against this candidate's theoretical mass
ion_score	always	Score A: ion-fitting fraction in [0, 1]
ion_hit_count	always	Integer count of ion-list fragments hit within tolerance
ion_hits_mz	always	Semicolon-separated m/z list of hits
score_b	Score B active	Float in [0, 1]
score_b_rank	Score B active	Per-scan rank by score_b (1 = highest)
score_b_selected	Score B active	Boolean; true for the per-scan top-ranked candidate with non-empty selected composition
selected_composition	Score B active	Chosen composition string for selected rows
score_b_motif_summary	Score B active	Short text rationale

Filename: {sample_name}_pseudolabels_{YYYYMMDD_HHMMSS}.tsv next to the converted CSV.

Trainable CSV (output of build_trainable_from_CGA or MAS merge)

Comma-separated. One row per spectrum.

Column	Notes
MS2scan_no	From scan CSV
protonatedmass	Optional; present when <i>Use protonated mass as a model feature</i> is enabled. Set to 0.0 for Non-glycan rows.
<ion_mz_1>, <ion_mz_2>, ..., <ion_mz_N>	Per-fragment feature columns. Cell value is $\log_{10}(I) + 1$ where I is the max intensity in the PPM window, or 1.0 if no peak matched. Column names are the reference ion m/z values formatted as floats.
Structure	Composition label (e.g., F1H5N4) for positive rows; literal Non-glycan for sampled negatives.
uid	Optional; populated when datasets have been combined via <i>Combine Datasets for Training</i> . Format: {origin_basename}#scan{scan_no}.

Filename in CGA path: {sample_name}_trainable_fromPL_{YYYYMMDD_HHMMSS}{_withMass|_noMass}.csv. In MAS path: {sample_name}_merged_{YYYYMMDD_HHMMSS}.csv.

Predictions CSV and supporting outputs

Each successful prediction run creates a folder named for the run (encoding feature-set and gate-state suffixes, e.g., {model_base}_MS1feat_PG10ppm/) next to the unlabeled input CSV. Folder contents:

- **predictions_with_conf.csv** — main prediction output. Columns: MS2scan_no, predicted_label, confidence (top-class probability), top1_proba, top2_proba, margin, plus passthrough columns from the unlabeled input (protonatedmass, theoretical_mass, ppm_precursor, pred_ok, uid when present), and per-class probability columns when the gate triggered the legacy proba expansion path.
- **class_summary.csv** — one row per predicted class with count, mean_confidence, and pass_rate (fraction passing both confidence threshold and precursor gate).
- **run_summary.json** — reproducibility manifest. Records model path, unlabeled input path, threshold settings (τ , margin, enable flag), precursor gate setting (enable flag, ppm), in-silico CSV path (when gate is on), method base name, GlycoMSP version, output file inventory.
- **run_summary.md** — human-readable rendering of run_summary.json. Same content, prose-formatted.
- **to_review_low_conf.csv** — spectra whose top-class probability fell below τ or whose margin fell below the configured margin. Worth manual review.
- **to_review_gate.csv** — spectra whose predicted composition disagreed with the measured precursor mass beyond gate_ppm. Includes ppm_error column.
- ***_gate_debug.csv**, ***_gate_passed.csv**, ***_gate_nolut_bycounts_debug.csv**, ***_PG_counts.csv** — gate-internal diagnostic outputs. Useful when investigating why specific spectra were demoted.

Trained model and sidecars (output of train_model)

File	Contents
<dataset_stem>_rf_model.skops	The trained RandomForestClassifier serialized via skops.
<dataset_stem>_labelencoder.joblib	Fitted LabelEncoder mapping class strings to integer labels.
<dataset_stem>_features.json	Ordered list of feature column names. Critical for predict-time alignment.
<dataset_stem>_training_run.json	Reproducibility manifest. See Section 4.5.

Models saved by older builds may use .joblib or .pkl extensions instead of .skops. The application’s loader handles all three; sidecar paths are derived from the model path with the appropriate extension.

5.5 Default parameters

A single reference table for every default value the application uses out-of-the-box. Source: code defaults (mspfileloaderv14_vars.md) and screenshots; manuscript-published values are flagged where they diverge.

Parameter	Default	Where set	Notes
Precursor PPM tolerance (CGA matching)	20.0 ppm	run_pseudolabeling ppm_value	Configurable via CGA Setup
Top-N candidates per scan (CGA)	3	run_pseudolabeling keep_top_n_per_scan	Caps candidates by smallest $\backslash ppm_error\backslash $
Ion-list PPM tolerance (Score A matching)	10.0 ppm	run_pseudolabeling ion_ppm	
Score A anchors required	2	run_pseudolabeling anchors_required	Currently a parameter but not consumed in body
Feature-extraction PPM tolerance	20.0 ppm	build_features_from_peaks_log10_plus1	Used at training and prediction

Parameter	Default	Where set	Notes
No-peak default cell value	1.0	feature extraction	$\log_{10}(1) + 0 = 0$ would lose info; +1 keeps cells distinguishable
Negative sampling: enabled	True	Negative Sampling Setting	
Negative sampling: ratio	1.0	neg_ratio_var	neg:pos cap
Negative sampling: min ion-hit gate	0	min_hits_var	Scans below this become negatives
Negative sampling: PPM tolerance	10 ppm	ppm_tol_var	
Ion mining: enabled	False	ion_suggest_enable_var	Opt-in
Ion mining: PPM tolerance	10.0 ppm	ion_suggest_ppm_var	
Ion mining: Da floor	0.030 Da	ion_suggest_dafloor_var	Low-m/z merge floor
Ion mining: min support	5	ion_suggest_minsupp_var	Per glycan class
Ion mining: top-K	60	ion_suggest_topk_var	Suggestions exported
Min samples per class	5	train_model	Drop classes smaller than this
Max negative cap	3× largest glycan class	train_model	Manuscript baseline; configurable
Test split	0.20	test_split_var	
Validation split	0.00	val_split_var	No validation fold by default
Stratified split	True	stratify_var	
RF n_estimators	400	n_estimators_var	
RF max_depth	None	RF defaults	Unrestricted depth
RF min_samples_split	2	RF defaults	
RF min_samples_leaf	1	RF defaults	
RF max_features	sqrt	RF defaults	
RF class_weight	balanced	class_weight_var	
RF random_state	42	fixed	
Confidence threshold τ	0.65	thresh_val_var	Only when threshold enabled
Confidence margin	0.05	margin_val_var	Only when threshold enabled
Threshold enabled	False	enable_thresh_var	Disabled by default
Feature encoding ppm (training)	10 ppm	MAS-merge / CGA→trainable	
Feature encoding ppm (prediction)	20 ppm	extract_ion_intensities	More permissive than training
Precursor gate (predict)	True	apply_pred_precursor_gate_var	Opt-out at prediction time
Precursor gate PPM	10.0 ppm	precursor_gate_ppm	
Validation policy: allow missing method	True	exp JSON	
Validation policy: allow hash mismatch	True	exp JSON	

5.6 Score B workbook structure (overview)

The Score B Excel workbook contains six sheets. The schema is sufficient detail for **identifying** that you have a workbook in the right shape; **authoring** a workbook from scratch requires reading `score_b_config_instructions.md` (manuscript supplementary).

Sheet (typical name)	Purpose
Glycotopes	Defines named glycotopes (motifs) used by all subsequent rules. Each row is one glycotope with its compositional signature and reference fragment masses.
Direct evidence rules	Rules that score positively when their listed fragments are observed. The unconditional support contributions.
Gated rules	Rules that fire conditionally — their support applies only when a specified precondition rule has fired. Used to encode evidence chains.
Composition consistency	Penalties subtracted when observed evidence contradicts the candidate's composition.
Unexpected evidence	Penalties subtracted when observed evidence is not accounted for by any expected motif rule applicable to the candidate.
Configuration / metadata	Workbook-level settings: derivatization tag, charge mode tag, version notes. The application validates these on workbook load.

Validation runs via `msp_CGA_structscore.ScoreBLoader.load`. Click **Validate Score B Workbook** in the CGA Setup window to surface load errors before running CGA analysis.

Part 6 — Configuration & advanced

For most users, accepting the defaults from Part 2 is enough. This Part covers the controls you reach for when the defaults don't match your data — typically because your dataset is unusually large, unusually small, drawn from a non-typical instrument configuration, or being prepared for a comparison study where parameter choices matter.

6.1 The ML parameters editor

The Train/Test Parameters dialog (Quickstart Step 2.7) covers the most common adjustments. For deeper configuration — anything beyond what the dialog exposes, or any parameter you'd like to version-control as a JSON preset — the application provides a richer editor.

From inside the Train/Test Parameters dialog, click the *Open params editor* button (or wire it via the Train Tab's *Set Parameters / Train the Model* button). This opens a 1200×600 modal with two side-by-side panels.

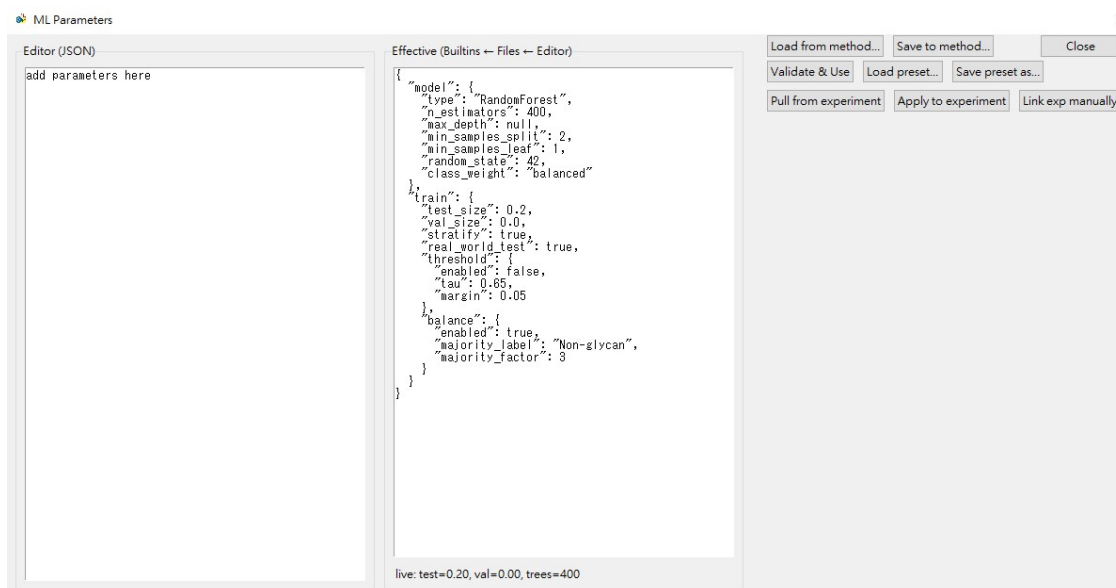


Figure 13. ML Parameters editor

Modal Toplevel, two panels. Left: editable JSON text area with header *Editor (your changes)*. Right: read-only *Effective (final, merged)* JSON preview, recomputed live as you type. Bottom toolbar: *Pull from exp*, *Apply to exp*, *Load preset...*, *Save preset as...*, *Link experiment JSON...*, *Load from method...*, *Save to method...*, *Validate & Use*, *Close*.

The merge model. The “effective” parameters used at training time are computed from up to four sources, layered:

1. Built-in defaults (BUILTIN_ML constant in source).
2. The experiment JSON’s `ml_defaults`, if linked.
3. The editor pane’s JSON, if non-empty.
4. The Train/Test Parameters dialog’s tk var values (when “Validate & Use” is clicked).

Each layer overrides the previous. The right-hand pane shows the final merged result so you can see exactly what the model will train with before you commit.

Buttons.

- **Pull from exp** — reads `ml_defaults` from the linked experiment JSON into the editor pane. Lets you start from a previously-saved exp-level baseline.
- **Apply to exp** — writes the editor JSON back into `ml_defaults` of the linked experiment JSON. Useful when an experiment-wide standard parameter set is what you want.
- **Load preset...** — picks any JSON file via dialog, loads its content into the editor (accepts both whole-exp payloads — `ml_defaults` is extracted — and plain ML-params files).
- **Save preset as...** — writes the current editor JSON to a file. Use for sharing parameters across labs or version-controlling them in git.
- **Link experiment JSON...** — re-binds the editor to a different `.exp.json` so subsequent Pull/Apply operate on it.
- **Load from method...** — reads `ml.parameters` from a method JSON.
- **Save to method...** — writes the merged effective parameters to a method JSON’s `ml.parameters` block (with a local-time `ml.updated` timestamp).
- **Validate & Use** — re-runs the merge with the editor as the rightmost layer, rebinds `effective_ml_params` (the closure variable read by the trainer), and persists into the in-memory ML state. The window does NOT auto-close after — you can verify, adjust, and re-validate as needed.

- **Close** — closes the editor without persisting (your changes since the last *Validate & Use* are lost).

Recommended workflow for parameter studies. Author a JSON preset with the variation under test, save it, link the exp JSON, click *Apply to exp*, then train. Repeat with the next preset. The exp JSON's `ml_defaults` always reflects what was used; the trained model's `training_run.json` records the same.

6.2 Precursor (MS1) gate

The **Precursor (MS1) Gate for Predictions** panel on the Predict tab (Quickstart Step 2.8) compares each spectrum's measured precursor mass against the theoretical mass of the predicted composition. Predictions where the measured-vs-theoretical PPM error exceeds the configured tolerance are demoted to the gate review queue.

When the gate helps. The gate is most useful for CGA-derived models where the labels were assigned at a known precursor PPM tolerance. If a prediction's composition doesn't agree with the precursor mass within a similar tolerance, something is off — either the model is wrong, or the spectrum's precursor was assigned to the wrong scan, or the in-silico CSV is incomplete. The gate surfaces these cases as a separate review queue rather than mixing them into the main predictions.

When to disable. Disable the gate when working with high-mass-error datasets (some instrument configurations don't reach 10 ppm reliably), when the in-silico CSV is known incomplete, or when running an exploratory prediction where you want the model's raw output regardless of mass agreement. Untick the *Apply precursor gate* checkbox; the application then sets `pred_ok = True` for all rows and skips the gate step.

Required input when enabled. The gate needs an in-silico CSV mapping composition → theoretical mass, set via the *Select in-silico CSV file* button. The same in-silico CSV used in CGA Setup (Step 2.4) works directly; if you trained on multiple CGA samples with different libraries, pick the library that covers the broadest composition range.

Gate vs confidence threshold. The two checks are independent. A prediction must pass *both* to land in the main output: - Confidence threshold: top-class probability $\geq \tau$ AND margin $\geq$ configured margin (or threshold disabled). - Precursor gate: $|\text{measured_mass} - \text{theoretical_mass}| / \text{measured_mass} \times 1e6 \leq \text{gate_ppm}$ (or gate disabled).

Predictions failing the threshold land in `to_review_low_conf.csv`. Predictions failing the gate land in `to_review_gate.csv` (with a `ppm_error` column). Predictions failing both land in *both* files. The output folder also contains `*_gate_debug.csv` and related diagnostic CSVs that surface the per-row gate decision rationale, useful when investigating cases that surprised you.

Counts-based fallback for composition lookup. When the in-silico CSV's composition column uses different formatting from the model's predicted labels (e.g., predicted K1 vs library KDN1), the gate's primary string-match lookup fails. The application then synthesises composition counts from the in-silico monomer columns (Hex, HexNAc, etc.) and tries a counts-based match. Spectra resolved this way appear in `*_gate_nolut_bycounts_debug.csv`. Spectra unmatched by either path are skipped (counted in the run summary, not silently lost).

6.3 Threshold sweeps and τ selection

The training step optionally produces a τ -sweep summary — accuracy / macro-F1 evaluated at multiple τ values on the held-out test fold. Use it to pick a τ for your prediction runs that balances coverage and precision for *your* dataset.

The general pattern: as τ rises, fewer spectra are predicted (coverage drops) but those that *are* predicted have higher confidence (precision rises). Macro-F1 typically peaks at a moderate τ — too low and confused predictions degrade per-class recall; too high and rare classes get demoted out of the main output entirely.

Practical rules of thumb.

- **Small datasets (< 100 spectra/class):** disable threshold demotion entirely. Set the *Apply confidence threshold* checkbox off. The model's probability calibration is too coarse for thresholds to be meaningful at this scale.
- **Mid-size datasets (100-1000 spectra/class):** enable the threshold and start near $\tau = 0.65$ (the GUI default value), then adjust based on the τ -sweep summary — use whichever τ maximises macro-F1.
- **Large datasets (> 1000 spectra/class):** thresholds become meaningful, and aggressive values ($\tau \geq 0.70$) start to make sense for production prediction runs where you'd rather discard uncertain predictions than mix them into final output.

Margin selection. The margin parameter (default 0.05) protects against close-call mispredictions where two classes are nearly tied in probability. Raise margin to 0.10 or higher when you have many compositionally similar classes (e.g., glycans differing only by one monosaccharide) and want the model to be conservative when probabilities are split. Lower margin is rarely useful — the default catches the most common close-call failure mode without being aggressive.

The τ and margin you settle on can be saved into the experiment JSON via the params editor's *Apply to exp* button so subsequent training runs pull the same values.

Part 7 — Troubleshooting & known limitations

This Part covers the most common issues users encounter, plus the limitations baked into the current version that are good to know upfront so they don't surprise you mid-analysis.

7.1 Common errors and their fixes

RAW conversion fails with “module not found” or similar. GlycoMSP needs the Thermo RawFileReader library plus the pymsfilereader Python package to read RAW files directly. This works only on Windows. If you're on macOS or Linux, convert your RAW files to mzML externally (e.g., MSConvert) and use **Add mzML File** instead.

RAW conversion fails with “could not write to output directory”. The conversion writes a temporary CSV in the chosen output folder before promoting it to the final filename. Check that the folder exists, that your user account has write permission, and that any antivirus / sync client (OneDrive, Dropbox) isn't holding the destination file open. Watch the application's log panel for the specific path that failed.

Conversion stops mid-batch with the editor stuck in red “Error:” state. The conversion thread caught an exception. The current file's batch advance is halted — the rest of the batch will not run automatically. Close the editor (it offers to force-close), then click **Save Log** on the main window and read the log for the underlying error message. Most often: corrupt RAW file, malformed metadata, or a temp-path collision.

On Windows, the application's console flashes briefly after an error then closes before you can read it. This is a known limitation of the Windows excepthook in the current build. Workarounds: (a) launch GlycoMSP from a persistent terminal (Command Prompt or PowerShell) so the console stays open after the application exits, (b) use **Save Log** before the

application closes, (c) the application also writes errors to the log file passed via Save Log. A future build will keep the error console open until acknowledged.

The “Convert Raw to CSV” button does nothing when only mzML files are selected on a system without pymzml. If the optional pymzml library isn’t installed, the application’s batch dispatcher currently treats the entire batch as un-convertible even if it contains only mzML files (which don’t actually need pymzml to convert). Install pymzml via pip even if you only plan to use mzML, until this is fixed in a future build.

Drag-drop in the Dataset Explorer doesn’t work on rows above the sample level. Drag-drop only reassigns Sample nodes between Experiment nodes. Dragging an Experiment node, a file slot, or a method reference is silently ignored. To rename or reorganise at the Experiment level, edit the experiment JSON directly or use the right-click menu on Sample nodes.

Right-click “Link and Validate Sample” appears greyed out. The Sample needs both a CSV and an Excel attached before validation can run. Check that both files are attached (visible as csv and excel slots on the sample node). Metadata is validated separately at stage 6 of the link-and-validate workflow; a missing metadata file prompts you to create one rather than blocking.

MAS validation fails on the Excel file with “Excel Validation Failed” but the dialog has no detail. The error title is shown but the dialog body is empty due to a current dialog-construction limitation. The actual error reason is logged to the main window log panel — open it (or click **Save Log**) and look for the matching [Excel] or [MAS] entries. The most common reasons: missing MSlist sheet, missing or non-numeric mass column in the Ionlist sheet, or unrecognized scan-id column header.

ML training fails with “Invalid Label” early-return. The Label Column setting in the Train tab points at a column that doesn’t exist in the CSV, or the column exists but contains no labels (all NaN). Verify the *Step 2: Select Label Column* combobox is set to the actual column name in your CSV (default is Structure).

Predictions look random — every spectrum gets the same class. Most likely cause: feature columns at predict time don’t align with the model’s training column order. Check that you’re using the same ion list at predict time as at train time. The application reorders columns to match `train_feats` from the model’s `_features.json`, but if your unlabeled CSV is missing too many columns, the model receives mostly the no-peak default (1.0) for everything and falls back to predicting the majority class.

Predict tab gate writes lots of `to_review_gate.csv` rows but no main predictions. The model is predicting compositions whose theoretical masses don’t match the measured precursor masses. Check: (a) is your in-silico CSV the right one for this dataset? (b) is the `gate_ppm` tolerance reasonable for your instrument? (c) is the dataset actually a different glycan class from what the model was trained on? Disable the gate temporarily to see what the model would predict without filtering.

macOS file dialogs occasionally crash the application during multi-file selection. A platform issue with tk on certain macOS versions. The application catches the underlying exception and degrades to the **Add MS2 CSV(s) / Add Excels** path. If repeated, prefer single-file selections.

7.2 Known limitations

N-glycans and O-GalNAc only. The in-silico library generators (`compv4.NGlanch` and `compv4.0Glanch`) cover N-glycans and O-GalNAc glycans. Other glycan types (O-mannose, glycolipids, GAGs) are not currently supported. Adding new glycan classes requires extending the composition generator’s flag schema and partition sets.

Single classifier in v1. The Train tab's *Choose Classifier* combobox lists Random Forest as the only option. The infrastructure (label encoding, balancing, splitting, threshold demotion) is classifier-agnostic; adding alternative classifiers (gradient-boosted trees, support vector machines, neural networks) is feasible but not in this build.

Multi-method tracking is partial. Each sample's `_methods` array can hold multiple method-JSON references (one per family — typically one MAS and one CGA), but the active-method slot `files["json"]` holds only the most recently saved one. UI surfaces (Export Method v1 buttons, exp-JSON writebacks) act on the active slot. If your workflow requires switching between MAS and CGA methods on the same sample, save each method explicitly and re-link before running the relevant pipeline.

Ion list must align across mixed-source training. Combining CGA-derived and MAS-derived rows requires matching feature spaces. The application's outer-join concatenation pads missing features with 1.0 (the no-peak default), so mismatched ion lists work but degrade signal. Use the same ion list across samples you intend to combine.

Score B is independent of MAS. The motif-aware re-ranking step operates on candidate compositions, of which MAS rows have only one (the expert label). Score B is therefore CGA-only. There is no analogue for refining expert labels with motif evidence.

Create unlabeled dataset of certain experiment is per-experiment. The current build runs this workflow against one experiment JSON at a time. Predicting across multiple experiments simultaneously requires running it once per experiment.

Saved CGA settings exclude OG core types. The *Save CGA settings* button writes the flag panel state to JSON, but for O-GalNAc runs the OG core-type selections (Core 1 / 2 / 3 / 4) are stored only in memory and are not part of the saved/loaded JSON. Re-load a saved O-GalNAc CGA preset and you'll need to re-select the core types. Future builds will close this gap.

Older models (.joblib, .pkl) load but cannot be re-saved with full provenance. The application loads `.joblib` and `.pkl` models for predict-time use, but the `training_run.json` reproducibility manifest is only generated during *training*. Models trained in older builds may lack the manifest; treat them as legacy artifacts and re-train when reproducibility matters.

Method JSON paths are stored OS-natively. Method JSON files written on Windows have backslash separators; macOS / Linux files use forward slashes. The application's read-time path resolver handles cross-platform load (Windows-drive regex), but diff-ing a method JSON across hosts shows path-separator changes that aren't semantic. Experiment JSON files use POSIX paths uniformly and diff cleanly across hosts.

Negative sampling state is global across pipelines. The Negative Sampling Setting dialog's controls affect MAS merge, CGA → Trainable, and CGA pre-mining negatives simultaneously. There is no per-pipeline override at the dialog level. Workaround: configure the dialog before each pipeline run, or use the JSON-level controls in the params editor.

Part 8 — Reproducing the manuscript results

This Part is for reviewers and readers who want to reproduce the results reported in the GlycoMSP manuscript. Each subsection maps to a package in **Supplementary Table S2c** (Zenodo), so you can find the exact input/output files. The reported analyses used a pre-release build of **GlycoMSP v1.10**.

Note. Package ID numbers below follow Supplementary Table S2c. The MAS-ML LOTO and pooled cross-validation analyses are bundled into a single package (ID 7); downstream IDs shift accordingly (GlycoGenius → ID 8, CandyCrunch → ID 9).

8.0 Quick start for reviewers

If you only do one thing, reproduce the **MAS-ML evaluation (Result 2)** — it is a single standalone script with bundled data and fixed seeds:

```
python mas_ml_loto_manuscript_analysis.py \
  --brain zf_sPerMeNG_brain_merged.csv \
  --intestine zf_sPerMeNG_intestine_merged.csv \
  --ovary zf_sPerMeNG_ovary_merged.csv \
  --features training_run.json \
  --gms-src /path/to/GlycoMSP/src \
  --outdir MAS_ML_LOTO_results
```

Expected (glycan-only macro-F1): within-tissue **0.931 / 0.935 / 0.952** (brain / intestine / ovary); leave-one-tissue-out **0.635 / 0.599 / 0.832**; pooled 5-fold CV **0.904 ± 0.038**. The brain within-tissue run reproduces the GUI rf_performance.txt exactly (test support 219, accuracy 0.99). The other results are GUI-driven workflows described below.

8.1 Setup and environment

- **Source code:** GitHub <https://github.com/henry4204aaa/GlycoMSPParser> (v1.10 tag, MIT License).
- **Install:** Python 3.10+ with pandas, numpy, scikit-learn, openpyxl, pymzml, joblib/skops (and pmsfilereader on Windows for RAW). See the repository requirements files.
- **Reanalysis data:** the Zenodo archive (DOI in the manuscript Availability section), organised into the packages listed in Supplementary Table S2c.
- **Reference environment:** the reported metrics were generated with scikit-learn 1.2.0; outputs were verified byte-identical on scikit-learn 1.7.2, so a current install reproduces the same numbers.
- **Paths caveat:** GlycoMSP method/experiment JSON files may store absolute paths. When reproducing on another machine, update the saved paths to your local working directory before loading them (see the Quick Start Guide bundled in the Zenodo Misc package).

8.2 MAS-ML evaluation — Result 2 (Supp S2c ID 7)

The MAS-ML held-out, leave-one-tissue-out (LOTO), and pooled cross-validation results are reproduced by the standalone script in manuscript_reproduction/ (also archived as Zenodo package ID 7 — the MAS-ML standalone, bundling held-out, LOTO, and pooled cross-validation). Run the command in §8.0 against the three zebrafish MAS trainable CSVs and the training_run.json from the package. The script reads GlycoMSP-exported trainable CSVs and reproduces the GUI pipeline without modifying core code; full design, metric definitions, and the GUI-equivalence evidence are in the package's MAS_ML_LOTO_technote.md and MAS_ML_LOTO_validation_report.md. Primary metrics are glycan-only (the operational Non-glycan class and out-of-label-space compositions are reported separately).

8.3 CGA / CGA-ML — Results 3-4 (Supp S2c ID 4c, 1c)

CGA/CGA-ML results are reproduced through the GUI using the provided in-silico glycan library, fragment-ion list, and CGA settings:

1. Convert the provided mzML/RAW to an MS2-indexed CSV (or use the bundled converted CSV).
2. In **CGA manager**, link the bundled in-silico CSV and ion list, then **Start CGA analysis**.
3. **CGA → Trainable** to build the trainable CSV, then **Train** and **Predict** as in Part 2.

For **U-937 O-GalNAc (Result 3, package 4c)** this recovers the nine MALDI-profile compositions plus the four CGA-ML-supported minor compositions (Fig. 3). For the **zebrafish brain F2H4N5 rescue (Result 4, package 1c)** the CGA-ML output recovers the omitted composition traceable to its MS2 scans (Fig. 4). CGA-only and negative-mode contexts correspond to Supplementary Figs. 2–3.

8.4 External-tool comparison — Result 5 (Supp S2c ID 8-9, Supp Table S3)

The GlycoGenius and CandyCrunch comparison (Fig. 5) is reproduced from the archived tool settings and reports: **GlycoGenius** (ID 8 — .ini/.gg project files + results; Supp S3a settings) and **CandyCrunch** (ID 9 — exact run commands + reports; Supp S3b). Reported values: MAS-ML recovered 19 of 20 eligible compositions; GlycoGenius re-identified all 38 imported MAS compositions; CandyCrunch overlapped 8 and produced 11 additional; in the larger search-space analysis GlycoMSP CGA assigned 573 (88 retained by CGA-ML), GlycoGenius reported 1042 (230 good-quality), and CandyCrunch reported 19.

The archived packages also include runtime/error observations and the minimal workarounds we applied, recorded as *issues encountered in our tested environment* (with OS and version details). These are provided for reproducibility and transparency, not as a judgement of the other tools.

Appendix A — Glossary

Terms specific to GlycoMSP, plus selected machine-learning and mass-spectrometry terms whose meaning may not be obvious from context. The manual assumes mass-spectrometry literacy and defines machine-learning terms in-prose at first use; this glossary is a backstop for forgetting.

Anchor (in Score A). A required reference fragment that must be present (above any intensity threshold) for the application to consider a Score A computation valid. The “anchor gate” prevents weak coincidental matches from inflating Score A. Default: 2 anchors required.

CGA — Constraint-based Glycan Annotation. GlycoMSP’s computed-labeling pipeline. Generates candidate glycan compositions from an in-silico library, filters by precursor mass agreement, scores by fragment-ion support (Score A), optionally re-ranks by motif evidence (Score B), and produces per-spectrum labels. Manuscript-canonical name. The end-to-end pipeline including ML training is named **CGA-ML**.

Class weight. A per-class multiplier applied during Random Forest training that compensates for class imbalance. The balanced setting computes weights inversely proportional to class frequency, so rare classes contribute proportionally more to the loss function during training.

Confidence. The probability of the top class assigned by the Random Forest’s predict_proba. Used by the threshold-demotion step at prediction time.

Constraint flags. User-supplied configuration to the in-silico library generator (compv4.NGLaunch / compv4.0GLaunch), specifying which glycan structural features to include or exclude in enumeration. Examples: hybrid-type N-glycans, bisecting GlcNAc, peripheral fucose, sialic-acid limits.

Derivatization. The chemical modification applied to glycans during sample preparation prior to mass-spec analysis (e.g., permethylation). The Derivatization Type metadata field tells GlycoMSP how to compute precursor mass shifts and which fragment-ion patterns to

expect. Recognised values include `PerMe`, `PerMe(Freeend)`, `PerMe(Reduced)`, `None`. Custom values are accepted with a manual mass-shift entry.

Feature matrix. The wide-format CSV consumed by the ML model. Rows are spectra; columns are reference-ion features (one per fragment in the ion list); cells are $\log_{10}(I) + 1$ where I is the max peak intensity within the PPM window for that fragment, or 1.0 when no peak matched.

Ion mining. The GUI procedure (button “Ion Mining”) that identifies m/z peaks appearing preferentially in glycan vs Non-glycan spectra, ranks them by lift, and outputs candidate diagnostic ions. (Informally a “feature-mining” step since it surfaces candidate ML features; the manuscript does not use that term.)

Gate. A binary filter applied to a prediction. The application has two: (1) the **confidence gate** (top probability $\geq \tau$ AND margin $\geq$ configured margin) and (2) the **precursor MS1 gate** (predicted-vs-measured mass error within `gate_ppm`). Both must pass for a prediction to land in the main output.

Glycotope. A structural sub-element of a glycan recognisable by characteristic fragmentation. Score B operates on glycotope-level evidence rather than whole-composition evidence, which is what makes motif-aware re-ranking possible.

In-silico library. A computationally generated catalog of theoretical glycan compositions consistent with user-specified constraints. Each entry has a name, a monosaccharide-count tuple, and a theoretical mass. Reusable across samples sharing derivatization and ionization mode.

Label encoder. A deterministic mapping of class strings (e.g., `F1H5N4`, `Non-glycan`) to integer labels (0, 1, 2, ...). Random Forest works internally with integer labels; the encoder is fitted at training time and saved alongside the model so prediction-time outputs can be decoded back to strings.

Manual Annotation (MA) / MAS. “MA” is the input format — an Excel workbook with `MSList`, `Ionlist`, `GlycanList`, `Information_Sheet` sheets. **MAS** is “MA-Supported” and refers to the pipeline that consumes MA inputs. The end-to-end pipeline including ML training is named **MAS-ML**.

Margin. The difference between the top-class probability and the second-class probability for a single prediction. Used as a conservative-confidence requirement: even if the top-class probability exceeds τ , a prediction is demoted if the margin (top – second) is too small. Default 0.05.

Motif. A structural pattern matched by Score B rules. Each glycotope defined in the Score B workbook corresponds to a motif; rules fire when the spectrum’s fragment evidence matches the motif’s diagnostic ions.

Precursor. The intact glycan ion that was selected and fragmented to produce an MS2 spectrum. Its measured mass (`protonatedmass`) is the basis for precursor-PPM matching against the in-silico library.

Pseudolabel. Original term for CGA-derived computed labels. Persists in the source code (file naming, function names like `run_pseudolabeling`) but is being phased out in favor of “constraint-based annotation” / “CGA result”. The manuscript and this manual use the new naming.

Random Forest (RF). An ensemble classifier consisting of many decision trees trained on bootstrap samples of the data with random feature subsets at each split. Predictions are the majority vote across trees. Probabilities (`predict_proba`) are the per-class fractions of trees voting for that class.

Score A. The ion-fitting score: proportion of an ion list’s reference fragments detected in the spectrum within PPM tolerance. Manuscript variable *a*. The primary candidate-selection score in CGA when Score B is inactive.

Score B. The motif-aware score: a [0, 1]-clamped combination of motif-level support minus composition-consistency and unexpected-evidence penalties. Manuscript variable *b*. Optional; enabled by attaching a Score B workbook in the CGA Setup window.

Stratification. Maintaining class proportions across train / validation / test splits. With *n* total spectra and a class with *k* examples, stratification ensures each split contains approximately $(k/n) \times \text{split_fraction}$ examples of that class. Avoids “test fold has no rare-class examples” pathologies.

τ (tau). The minimum top-class probability for a prediction to pass the confidence gate. Default value 0.65, with the gate **disabled by default** in v1.10. Predictions below τ are demoted to the review queue when the gate is enabled.

Trainable CSV. The wide-format CSV that ML training consumes — schema in Section 5.4. Both CGA and MAS pipelines produce this same shape so models can train on a mix.

Training / validation / test split. The standard ML data partition. Training data fits model parameters; an optional validation fold can tune hyperparameters and select τ during the τ -sweep; test data is held out entirely until final reporting. The GlycoMSP v1.10 default is an **80 / 20 train / test split** (`test_split = 0.20`, `val_split = 0.00` — no validation fold by default), matching the manuscript; raise `val_split` only if you want a validation fold for τ tuning.

Unlabeled dataset. A feature-extracted CSV without Structure labels. The input to prediction. Built either from scratch via *Load unlabeled dataset* or via *Create unlabeled dataset of certain experiment* which traverses an experiment JSON and produces one unlabeled CSV per sample.

WYSIWYG (in CGA design). The manuscript’s framing for CGA’s enumeration philosophy: the in-silico library encodes only structural motifs the user expects to observe in their samples (“what you see is what you get”), avoiding reliance on glycogene databases that may be incomplete or organism-specific.

Appendix B — Keyboard and right-click reference

GlycoMSP is primarily mouse-driven, but a handful of keyboard and right-click conventions speed up routine work.

Dataset Explorer (Prepare Dataset window)

Right-click on a Sample node opens a context menu with sample-level actions:

- *Link and Validate Sample* — runs the six-stage MAS validation (Section 3.1).
- *Merge Sample* — runs MAS merge (greyed out unless validation has succeeded).
- *Open Metadata Editor* — opens the Metadata Editor window for this sample, in metadata-only mode (skips the conversion step).
- *Remove* — removes the sample from the experiment. **Bookkeeping only — does not delete files on disk.** The CSV / Excel / metadata files remain in their folders; you can re-add them via the *Add* buttons.

Right-click on a file slot inside a Sample offers a *Remove* action that clears just that slot (e.g., remove the attached Excel without removing the CSV). Same bookkeeping-only semantics.

Right-click on an Experiment node offers no context-menu actions in the current build; experiment-level operations go through the bottom toolbar (*Set Experiment Method File*, *Save .exp.json*, etc.).

Drag-and-drop reassigns a Sample between Experiments. Click and hold on a Sample row, drag onto a different Experiment node, release. The Sample moves; its files and method references move with it. Drag-targets must be Experiment nodes; dropping on another Sample, on a file slot, or anywhere outside the Dataset Explorer is silently ignored.

The *macOS variant* binds right-click to both Button-3 and the macOS-conventional Button-2 plus Control-Button-1, so the right-click context menu works regardless of mouse hardware.

Modal dialog conventions

Most modal dialogs (CGA Setup, Train/Test Parameters, Negative Sampling Setting, Ion Mining, ML Parameters editor) follow standard Tk conventions:

- **Tab / Shift-Tab** — move focus across input widgets.
- **Esc** — close the dialog. For most dialogs this is equivalent to clicking *Close* without saving.
- **Enter** — when focus is on a button, presses it. Combobox fields don't intercept Enter, so it cascades to the dialog's default button (typically the most prominent action).
- **Combobox quick-navigation** — typing the first letter of an entry jumps to the first matching value. Helpful for the Glycan Type / Mass Analyzer charge mode / Derivatization Type dropdowns.
- **grab_set() modal style** — most dialogs grab focus, so clicks on the parent window are ignored until the dialog closes. If you find a window unresponsive, check whether a modal dialog is open behind it.

Main window

No keyboard shortcuts are bound. Buttons (*Add Raw File*, *About*, *Save Log*, *Convert Raw to CSV*, *Prepare Dataset*, *Run ML Analysis*) are click-only. Closing the main window prompts a confirmation dialog before exiting.

Ion Suggestions Viewer

In the *Browse Ion Mining result* viewer:

- **Click a row** to select it.
- **Copy selected m/z** copies the selected row's first column (m/z value) to the clipboard. Useful for pasting into your ion-list Excel.
- **Open folder** opens the OS file manager at the suggestions CSV's parent directory, so you can audit the full CSV outside the viewer.

Appendix C — Pointers to related documents

- **Manuscript (v1.0)** — the canonical source for naming (Score A / Score B, CGA / MAS, CGA-ML / MAS-ML), the Score B formula $b = \max(0, \min(1, \text{support} - \text{penalty_comp})$

- `penalty_unexpected`)), the “Score B preferred, Score A fallback” rule, and published parameters (RF 400 trees, `max_depth=None`, `class_weight="balanced"`, 80/20 train/test). Note: the manuscript does **not** define “feature mining/FM” — see §4.7.
- **Supplementary information (v0.99)** — datasets and file mapping (Supp Table S2), external-tool settings and reports (Supp Table S3), and the short Score B overview (Supplementary Data 2).
- **Score B detailed design** — *to be produced* (a re-analysis of the current `msp_CGA_structscore.py` Score B implementation). The reference workbook example lives in the active repo at `templates/GlycoMSP_scoring_update_example_v7.xlsx` (to be renamed for Score B at the v1.10 freeze).
- **Wiki-doc (developer reference, repo docs/)** — `docs/index.md` (L1 overview), `docs/indexes/mspfileloaderv14.md` (L2a/L2b function and variable maps), `docs/modules/<module>_docs.md` (L3 function references), and `routes.md` (data-flow for both pipelines + scoring). Cite section + line when raising code-level reports. (*Line numbers may lag the source — see the doc headers.*)
- **MAS-ML LOTO reproduction** — `manuscript_reproduction/`: the standalone script, `MAS_ML_LOTO_technote.md`, and the validation report; also archived in the Zenodo MAS-ML package (ID 7).

This PDF was converted and exported from the source Markdown by Claude (Cowork), with revisions requested and confirmed by the author. It is provided as a reference copy in both the Zenodo archive and the GitHub repository. The canonical source Markdown is maintained on GitHub and will continue to be updated with future maintenance and version releases.